

Degree Project in Electrical Engineering, specializing in Communication Systems,

Second cycle, 30 credits

Constraint Programming for Cloud-RAN: Modeling and Complete Enumeration with Constraint Programming SAT solver

SIYAN CHEN

Constraint Programming for Cloud-RAN: Modeling and Complete Enumeration with Constraint Programming SAT solver

SIYAN CHEN

Master's Programme, Communication Systems, 120 credits
Date: December 20, 2025

Supervisor: Issac Skog, Bin Li
Examiner: Slimane Ben Slimane

Host company: Ericsson AB
Swedish title: Begränsningsprogrammering för orkestrering av Cloud-RAN-resurser: Modellering och fullständig enumerering med Constraint Programming SAT solver

© 2025 Siyan Chen

Abstract

The rapid growth of network complexity and the increasing demand for flexible infrastructure present significant challenges for automated resource allocation in industrial network systems. Traditional rule-based methods or single-solution optimizers often struggle to manage the combinatorial explosion of device configurations and lack support for dynamic changes or multiple feasible options.

This study aims to model the network equipment combination problem as a constraint satisfaction problem (CSP) and address it using state-of-the-art constraint programming techniques. We concentrate on generating all valid, resource-optimal combinations of potential network nodes while minimizing the utilization of additional shared devices.

We implement a CSP model utilizing Google OR-Tools' constraint programming SAT solver (CP-SAT) solver and propose an incremental graph decomposition strategy that partitions the global problem into subgraphs based on connectivity. Each subproblem is solved independently, with their solutions subsequently merged through an efficient deduplication mechanism. Additionally, a mixed integer programming (MIP) version is implemented for comparative analysis.

Experimental results indicate that our CP-SAT approach significantly outperforms the MIP model in terms of scalability and solution diversity, particularly in large-scale network scenarios. The method can efficiently enumerate hundreds of optimal configurations within seconds.

This work contributes a practical, scalable, and dynamic CSP-based optimization framework for network configuration, which can be extended to other domains involving complex resource allocation and combinatorial design.

Keywords

Constraint Programming, CP-SAT, RAN Orchestration, Cloud-RAN, Service Function Chaining, Lab-as-a-Service, Optimal Enumeration

Sammanfattning

I testmiljöer för Cloud-RAN är förmågan att enumerera alla optimala konfigurationer särskilt värdefull för flexibilitet, resiliens och reproducerbarhet—men de flesta metoder för placering av virtuella nätverksfunktioner och Service Function Chaining skalar antingen dåligt eller returnerar endast en enda lösning.

Denna avhandling behandlar laboratorieorkestrering som ett komplett sökproblem och tillämpar Constraint Programming med CP-SAT-lösaren (OR-Tools) som ett alternativ till klassiska angreppssätt. Vi föreslår en tvåstegsmetod som först minimerar antalet ytterligare enheter som krävs och därefter enumererar alla lösningar vid den kostnaden. För att uppnå skalbarhet utformar vi en modulär mikrotjänst som kombinerar grafdekomposition, parallell lösning och online-avduplikering.

Omfattande experiment i realistiska laboratoriemiljöer visar att CP-SAT unikt kombinerar uttrycksfull constraint-modellering, inbyggd enumerering av flera lösningar och praktisk prestanda. Jämfört med Mixed-Integer Programming, klassisk CP och en djupförst-baserad metod enumererar CP-SAT konsekvent alla optimala lösningar samtidigt som körtid och minnesanvändning reduceras.

Resultaten visar att CP-SAT erbjuder en tillförlitlig, leverantörsneutral grund för automatiserad orkestrering i testbäddar för disaggregerade Radio Access Networks. Mer allmänt bidrar arbetet med en återanvändbar metodologi för komplett optimal enumerering i kombinatoriska optimeringsproblem med heterogena logiska begränsningar.

Acknowledgments

I would like to express my deepest gratitude to my supervisors, Professor Issac Skog and Bin Li, for their invaluable guidance, constructive feedback, and unwavering support throughout this journey. Special thanks also go to my examiner, Professor Slimane Ben Slimane, whose insightful comments significantly sharpened both the methodology and the presentation of this work. I am equally indebted to my colleagues at Ericsson AB for the stimulating discussions and for providing the laboratory environment that made these experiments possible. To my family and friends: thank you for your endless patience, encouragement, and the steadfast support that sustained me across the finish line.

Finally, I wish to dedicate a personal space to acknowledge my own journey. Moving to a new country to pursue this Master's degree was a leap of faith that has fundamentally reshaped my character and perspective. I am profoundly grateful for the resilience and courage I found within myself when navigating the "combinatorial explosion" of challenges. To the version of me that maintained academic excellence while embracing a new culture: thank you for your persistence, for the grace shown under pressure, and for the strength to balance every transition with unwavering determination.

Stockholm, December 2025
Siyan Chen

Contents

1 Introduction **1**
1.1 Background . . . 2
1.2 Problem . . . 3
1.2.1 Research Question . . . 3
1.2.2 Hypothesis . . . 3
1.3 Goals . . . 4
1.4 Research Methodology . . . 4
1.4.1 Experimental Design . . . 5
1.5 Delimitations . . . 5
1.6 Contributions . . . 6

2 Background **7**
2.1 The RAN Orchestration Challenge . . . 7
2.1.1 The Paradigm Shift of Network Virtualization and Softwarization . . . 7
2.2 Problem Formulation and Complexity . . . 8
2.2.1 Mapping RAN Scenarios to VNF Placement and SFC 8
2.2.2 Computational Complexity . . . 9
2.3 A Review of Solution Paradigms . . . 9
2.3.1 Exact Optimization via Mathematical Programming . 10
2.3.2 Heuristics and Metaheuristics . . . 12
2.3.3 Learning-Based Approaches for Dynamic Environments 14
2.4 Research Gaps in RAN Orchestration . . . 16
2.4.1 Gap 1: Lack of Multi-Solution Enumeration . . . 16
2.4.2 Gap 2: Inflexible Constraint Modeling . . . 17
2.5 Proposed Approach Based on Constraint Programming . . . 18
2.5.1 Expressiveness of Constraints . . . 19
2.5.2 Native Support for Multiple Solutions . . . 19
2.5.3 State-of-the-Art Performance and Scalability . . . 20

2.6 Synthesis and Comparative Analysis 20

3 Method **23**
3.1 Research Paradigm and Overall Approach 23
3.2 Experimental Design 24
3.2.1 Experimental Environment 24
3.2.2 Scenarios and Workload Templates 25
3.2.3 Baseline Methods 26
3.2.4 Evaluation Metrics 26
3.2.5 Reproducibility and Fairness 26
3.2.6 Data Collection and Sampling 27
3.2.7 Software Stack and Solver Configuration 27
3.2.8 Execution Protocol and Instrumentation 28
3.3 Formal Optimization Model 28
3.3.1 Sets and Parameters 29
3.3.2 Decision Variables 29
3.3.3 Constraints 29
3.3.4 Objective Function 30
3.3.5 Discussion 30
3.3.5.1 Linearization and Numerical Stability 30
3.3.5.2 Multi-Solution Enumeration Mechanisms . 31
3.4 Algorithmic Strategy for Scalability 31
3.4.1 Problem Decomposition 32
3.4.2 Solver Instantiation 32
3.4.3 Systematic Solution Enumeration 32
3.4.4 Scalability Considerations 32
3.4.5 Summary 33
3.5 Validity and Reliability 33
3.5.1 Internal Validity 33
3.5.2 External Validity 33
3.5.3 Construct Validity 34
3.5.4 Reliability 34
3.5.5 Summary 35
3.6 Summary of Methodology 35

4 Design and Implementation **37**
4.1 System Architecture and Core Algorithms 37
4.1.1 Architecture Overview 37
4.1.2 Model-to-code mapping 38

4.1.3 Two-phase optimal enumeration 39
4.1.4 Component decomposition and parallel solving 39
4.1.5 Deduplication and Merging of Solution Sets 40
4.2 Correctness and Complexity 40
4.2.1 Correctness of Decomposition 40
4.2.1.1 Decomposable constraints. 40
4.2.1.2 Decomposable objective. 40
4.2.1.3 Implication. 41
4.2.2 Equivalence under deduplication 41
4.2.3 Computational Complexity and Resource Usage 41
4.3 Configuration and Reproducibility 42
4.3.1 Solver Configuration 42
4.3.2 API Schema and Example 42
4.3.2.0.1 Example Request. 42
4.3.2.0.2 Example Response. 42
4.4 Robustness and Failure Handling 43
4.4.1 Timeout Handling 43
4.4.2 Input Validation 43
4.4.3 Standardized Error Codes 44

5 Results and Analysis **45**
5.1 Correctness of Results 45
5.2 Runtime Performance 47
5.2.1 Overall Scaling 47
5.2.2 Performance versus. Solution Space Size 48
5.2.3 Quantifying the Performance Gap Through Speedup Analysis 49
5.2.4 Tractability Analysis Under Extreme Conditions . . . 52
5.3 Memory Efficiency 53
5.4 Scalability and Decomposition 53
5.4.1 Why Decomposition is Beneficial 54
5.4.2 CPSAT versus DFS in the Context of Independence . . 54
5.5 Phase-wise Runtime Breakdown 55
5.6 Summary of Results 58
5.7 Reliability Analysis 60
5.8 Validity Analysis 61

6 Discussion **63**

7 Conclusions and Future work **65**
7.1 Conclusions 65
7.2 Limitations 66
7.3 Future work 67
7.3.1 Unaddressed Aspects of the Problem 67
7.3.1.1 Cost and Multi-Objective Analysis 68
7.3.1.2 Integration into a Dynamic System 68
7.3.2 Immediate Directions for Subsequent Research 68

References **69**

List of Figures

2.1 This figure illustrates Scenario A, in which a centralized baseband unit pool serves multiple RRHs over a constant-bit-rate CPRI fronthaul implementing functional split 8. 10
2.2 This figure illustrates Scenario B, an advanced disaggregated and virtualized RAN architecture in which the baseband unit is split into a virtualized distributed unit and a centralized unit, with functions deployed as software on general-purpose processors and radio units connected over packet-based fronthaul using eCPRI. 11

3.1 An overview of the research methodology, which combines formal modeling, algorithm design, and experimental validation in a lab-based Cloud-RAN testbed environment. 24

4.1 Modular architecture of the microservice and orchestration workflow. 38

5.1 Runtime scaling with number of nodes (linear scale). 49
5.2 Runtime scaling with number of solutions (log-Linear scale). . 50
5.3 Speedup (depth-first search (DFS) runtime / CP-SAT runtime) as a function of the number of optimal solutions (log-linear scale) 51
5.4 Extreme cases: DFS vs CP-SAT runtimes with percentage time saving. 52
5.5 Boxplot of memory savings compared to DFS. 54
5.6 Time savings as a function of number of subgraphs 55
5.7 Phase-wise runtime composition of the DFS baseline, showing how the relative contributions of Step 1, Step 2, and Step 3 evolve with increasing node count, together with the corresponding total runtime indicated by the black line 58

5.8 Phase-wise runtime composition of the CP-SAT solver, highlighting the dominance of Step 2 in the overall solving process and the comparatively stable total runtime across different node sizes 59

List of Tables

2.1 Comparison of solution paradigms across key dimensions. . . 22

3.1 Hardware configuration of the host machines (nodes) used in the testbed. 25
3.2 Kubernetes container resource quotas assigned to orchestration microservices. 25
3.3 Software stack and solver versions. 27

4.1 Key solver parameters and environment settings used across all experiments. 42

5.1 Comparison of solution counts between DFS and CP-SAT . . 47
5.2 Key data points for runtime versus number of solutions. 50
5.3 Strategic differences between DFS and CPSAT pipelines. . . . 57

List of acronyms and abbreviations

BBU	baseband unit
C-RAN	centralized radio access network
CI/CD	continuous integration and continuous deployment
CP	constraint programming
CP-SAT	constraint programming SAT solver
CSP	constraint satisfaction problem
CU	centralized unit
DFS	depth-first search
DU	distributed unit
GA	genetic algorithm
ILP	integer linear programming
LaaS	Lab-as-a-Service
MILP	mixed-integer linear programming
MIP	mixed integer programming
NFV	network function virtualization
O-RAN	open RAN
OPEX	operational expenditure
RAN	radio access network
RL	reinforcement learning
RRH	remote radio head
RU	radio unit
SFC	service function chain
vCU	virtual centralized unit
vDU	virtual distributed unit
VNF	virtual network function

VNS	variable neighborhood search
vRAN	virtual radio access network

Chapter 1

Introduction

The rapid softwarization of mobile networks is transforming the radio access network (RAN) from closed, appliance-centric systems into disaggregated and virtualized deployments such as centralized radio access network (C-RAN), virtual radio access network (vRAN), and open RAN (O-RAN). This evolution enables flexibility and vendor diversity, but it also introduces significant orchestration challenges [1]. For each service request, operators are required to select compatible virtual and physical components—such as a virtual distributed unit (vDU), a virtual centralized unit (vCU), fronthaul switches, and media converters. They must also place these components onto constrained infrastructure and interconnect them while adhering to stringent latency and bandwidth requirements. In disaggregated RAN test environments, this manifests as the repeated construction of valid and reproducible test topologies using heterogeneous devices under policy and capacity constraints [2]. Manual configuration is slow and error-prone, making it incompatible with the rapid iteration demanded by DevOps and continuous integration and continuous deployment (CI/CD) workflows.

This thesis formulates the orchestration challenge as a *complete search problem*, aiming to efficiently enumerate *all* optimal combinations of laboratory resources that satisfy RAN test constraints. Our approach is grounded in formal modeling and implemented as a cloud-native microservice. We argue that modern constraint programming (CP)—specifically the constraint programming SAT solver (CP-SAT) solver from Google OR-Tools—offers the ideal balance of modeling expressiveness, scalability, and native support for multi-solution enumeration [3][4].

From an industrial standpoint, the immediate need is to automate lab orchestration by rapidly producing workable configurations. From

an engineering and scientific standpoint, the deeper question is whether we can *systematically enumerate all feasible or optimal solutions* under heterogeneous constraints—something that existing approaches struggle to achieve at scale. Classical integer linear programming (ILP)/mixed-integer linear programming (MILP) guarantee optimality but are rigid and scale poorly, especially when tasked with enumerating many solutions. Heuristic and reinforcement learning (RL) methods can scale, but they sacrifice completeness and cannot enumerate multiple optima. CP strikes a middle ground: it supports rich logical constraints and exact search, yet has historically lacked industrial-scale validation. This thesis reframes the orchestration challenge as an empirical study of complete enumeration under constraints, contributing methodology and evidence for its feasibility.

1.1 Background

The evolution of mobile networks towards 5G and beyond has reshaped RAN architectures. Two representative scenarios illustrate this transformation [2]:

(A) The first model is a foundational centralized radio access network. In this setup, multiple remote radio heads, which handle the radio signals, connect to a shared, central pool of baseband units for signal processing. These connections rely on dedicated ”fronthaul” links using protocols like the common public radio interface (CPRI), which have very strict requirements for delay and timing. While this C-RAN architecture allows for efficient resource pooling, its high cost and the inflexibility of the fronthaul links are significant limitations.

(B) A more advanced model is the virtualized radio access network. This architecture breaks the functions of the traditional baseband unit into smaller software components: the radio unit, the virtualized distributed unit, and the virtualized centralized unit. These components run on standard cloud infrastructure and are connected using standard Ethernet for both ”fronthaul” and ”midhaul” links. By replacing specialized hardware with software on general-purpose servers, the vRAN offers much greater flexibility, but at the cost of much higher complexity in network management and orchestration.

Building on these trends, C-RAN emphasizes cloud-native technologies—containers, orchestration, and Infrastructure-as-Code—for elastic RAN deployments. Meanwhile, O-RAN promotes open interfaces and multi-vendor interoperability, further amplifying heterogeneity. Together, these developments create a multi-layered combinatorial dilemma, particularly acute in Lab-as-a-Service (Lab-as-a-Service (LaaS)) environments, which

form the core context of this thesis.

1.2 Problem

The resource allocation problem in disaggregated RAN environments is a complex combinatorial optimization challenge, closely related to virtual network function (VNF) placement and service function chain (SFC) mapping, both known to be NP-hard [5]. A critical review of the state-of-the-art highlights two primary research gaps:

- **Focus on singular optimality:** Most methods for RAN resource orchestration produce only a single optimal or near-optimal resource configuration. Even when solution pools are supported (e.g., in MILP solvers), they scale poorly and cannot guarantee a systematic enumeration of all optima. Yet in RAN test labs, multiple valid topologies are desirable for flexibility, resilience, and operator choice.
- **Weakness in modeling heterogeneous constraints:** Traditional optimization frameworks struggle to capture hardware-specific capabilities, logical exclusivity rules, and interdependencies. Encoding these in linear form is cumbersome or infeasible, while heuristics and ad-hoc logic lack generality [6].

1.2.1 Research Question

These gaps motivate the following central research question of this thesis:

> *How can all valid and optimal combinations of laboratory resources be efficiently enumerated under complex, heterogeneous constraints in large-scale disaggregated RAN test environments?*

1.2.2 Hypothesis

We hypothesize that complete enumeration of optimal resource allocations can be achieved with practical scalability by combining:

- A two-phase CP-SAT approach, which first determines the minimum number of additional switching devices and then enumerates all solutions achieving this optimum.

- Graph decomposition and incremental merging strategies, which break the problem into smaller subproblems and combine results efficiently, enabling scalability within containerized environments.

1.3 Goals

The overarching goal of this thesis is to design, implement, and evaluate a scalable microservice for automated orchestration in disaggregated RAN test environments, capable of enumerating all optimal configurations. This goal is divided into three aims:

1. **Optimization model:** Develop a robust constraint programming model that accurately captures heterogeneous constraints in a LaaS environment while minimizing the reliance on additional switching devices.

2. **Performance evaluation:** Assess the runtime, memory usage, and multi-solution enumeration capabilities of CP-SAT against depth-first search (DFS), ILP/MILP, and classical CP baselines across representative lab instances.

3. **System implementation:** Design and deploy the solution as a modular, containerized microservice that leverages graph decomposition, parallel solving, and online de-duplication for scalability and reproducibility within a Kubernetes-managed testbed.

1.4 Research Methodology

This thesis follows an engineering-driven, model-based methodology. The orchestration problem is formally modeled and solved through multiple solver-based implementations. Rather than relying on theoretical analysis alone, we implement and evaluate computational models in realistic problem settings. This approach is suitable for operational optimization tasks where empirical evidence is necessary to assess feasibility and performance.
The orchestration problem is formulated as a constraint-based optimization task. Each node in a test topology must be assigned a valid resource group such that the network connection constraints are satisfied, and the total number of extra switching devices is minimized. This naturally maps to a constraint satisfaction problem (CSP) or a mixed-integer programming (MIP) model.

To evaluate the effectiveness of different solving strategies, we implement the same problem using four different solvers:

- **CP-SAT (Google OR-Tools):** A constraint programming solver that supports efficient multi-solution enumeration. Our main implementation uses a two-phase strategy: first minimizing extra device usage, then enumerating all optimal configurations.
- **MILP (Gurobi):** A mixed-integer linear programming model solved via Gurobi. While it guarantees global optimality, it suffers from poor scalability and limited logical expressiveness.
- **Choco Solver:** A classical constraint programming solver used to compare performance with CP-SAT. It does not support multi-solution enumeration natively.
- **Depth-First Search (DFS):** A baseline brute-force method that attempts all combinations with pruning. It is used primarily to establish feasibility and verify correctness on small instances.

We select CP-SAT as the primary approach due to its ability to combine expressive constraints, complete search, and competitive scalability. Pure ILP and heuristic approaches are excluded from detailed analysis due to their lack of completeness or scalability for multi-optimum enumeration.

1.4.1 Experimental Design

Experiments are executed in a controlled Kubernetes environment with fixed resource quotas (2 CPU, 1.5 GiB). Each experiment is repeated $R = 20$ times, with 95% confidence intervals. The workload spans typical lab templates and extreme-load cases where DFS runtime exceeds 30 seconds. Metrics include runtime, memory, and the number of optima enumerated, enabling reproducible and fair cross-solver comparison.

1.5 Delimitations

This study focuses on disaggregated RAN test environments under the LaaS paradigm. The scope is defined as follows:

- **Objective:** Minimization of additional switching devices, with enumeration of all solutions achieving this optimum. Other objectives (e.g., energy, throughput) are excluded.

- **Environment:** Limited to controlled testbeds, not live production networks with fault tolerance and ultra-low latency constraints.
- **Architecture:** Operates within predefined vRAN/C-RAN architectures; new splits or designs are not addressed.
- **Abstraction:** Vendor-specific hardware/software details are abstracted into model constraints.
- **Parallelism:** Limited to two workers due to container quotas; this is an experimental constraint, not a methodological one.
- **Dynamics:** The focus is on static allocation; real-time re-orchestration under failures or traffic variation is outside scope.

1.6 Contributions

This thesis makes the following contributions:

- **Problem formulation:** Formalization of disaggregated RAN orchestration in LaaS as a complete search problem requiring enumeration of all optimal solutions.
- **Two-phase CP-SAT method:** A novel two-stage approach that minimizes additional switching devices and enumerates all optimal solutions at that cost.
- **Microservice design:** A modular, containerized orchestration microservice integrating graph decomposition, parallel solving, and online de-duplication for scalability.
- **Experimental validation:** Extensive evaluation on 42 lab instances, showing CP-SAT's unique ability to combine modeling expressiveness, efficient multi-solution enumeration, and runtime performance relative to DFS, MILP, and classical CP.
- **Engineering evidence:** Reproducible experiments, methodological insights, and guidelines for applying CP to network orchestration challenges.

Chapter 2

Background

This chapter surveys network resource orchestration with a focus on disaggregated RAN testbeds, establishing both the theoretical basis and the practical motivation for this thesis. We position the problem of automated resource allocation for a lab environment within network virtualization trends via two representative scenarios: a baseline C-RAN (Scenario A) and an advanced, disaggregated vRAN (Scenario B).

We then review predominant solution paradigms and critically assess their suitability. Two gaps emerge: (i) lack of multi-solution enumeration; and (ii) limited expressiveness for complex logical constraints—motivating our CP approach based on CP-SAT.

2.1 The RAN Orchestration Challenge

The telecommunications landscape is undergoing a shift toward softwarization and virtualization [7], now reaching the Radio Access Network (RAN). This promises flexibility but amplifies orchestration complexity in lab/testbed contexts due to hardware heterogeneity and hard timing constraints [8, 9].

2.1.1 The Paradigm Shift of Network Virtualization and Softwarization

The foundational concept is the centralization of baseband processing (Scenario A). A classic C-RAN places a baseband unit (BBU) pool at a central site serving multiple remote radio head (RRH) over constant bit-rate CPRI fronthaul (functional split 8). A key challenge is mapping RRH to BBUs while meeting strict fronthaul latency/bandwidth constraints [9].

However, the industry's progression towards 5G and Open RAN further advances this paradigm, culminating in a more complex architecture represented by our second model, Scenario B: The Advanced Disaggregated and vRAN Model. This sophisticated model embodies the principles of disaggregation and network function virtualization (NFV) [10]. In this scenario, the traditional monolithic BBU is deconstructed into modular and interoperable components. This architecture has four key characteristics. First, the baseband unit is disaggregated into a virtualized distributed and a centralized unit, which allows flexible deployment. Second, functions are virtualized using containers or virtual machines running on general-purpose processors managed by a telco cloud. Third, the radio units are heterogeneous in bandwidth and feature sets, requiring careful orchestration. Finally, packet-based fronthaul is used, based on eCPRI over Layer 2/3 equipment with precise time synchronization.

This transition triggers a combinatorial explosion: the orchestrator must choose functions, placements, and routes. Roughly, the search space scales with $|\mathcal{F}| \times |\mathcal{H}|^{|\mathcal{F}|} \times |\mathcal{P}|^{|\mathcal{L}|} \times |\mathcal{R}|$, where $\mathcal{F}$ are virtual network function (e.g., virtual distributed unit/virtual centralized unit), $\mathcal{H}$ hosts, $\mathcal{P}$ admissible paths per link set $\mathcal{L}$, and $\mathcal{R}$ radio unit (RU) [10, 11].

2.2 Problem Formulation and Complexity

The resource allocation challenge inherent in our scenarios, while distinctive in its specific constraints, exemplifies a broader category of problems that have been extensively examined within networking research. By correlating our practical issue with its theoretical foundations, we can anchor this thesis in a substantial body of established literature, thereby enabling us to leverage and critically assess existing solution paradigms.

2.2.1 Mapping RAN Scenarios to VNF Placement and SFC

The primary function of our orchestrator is analogous to two well-established challenges in network virtualization: VNF Placement and SFC mapping[12].

VNF Placement is formally defined as the problem of allocating a required set of VNF to a collection of physical hosts, such as servers within a data center. This allocation must adhere to the resource capacity constraints imposed by the hosts and typically aims to optimize specific objectives, such as minimizing deployment costs[13].

SFC expands upon this concept by introducing sequential dependencies. An SFC delineates an ordered sequence of VNFs that a particular traffic flow must traverse in order to receive comprehensive service.

In our RAN scenarios, the resource allocation problem can be modeled as a variant of SFC mapping. This analogy becomes increasingly salient when transitioning from Scenario A to Scenario B:

- In Scenario A, the challenge represents a simpler case of physical node mapping: connecting RRH to a centralized physical BBU pool.
- Conversely, Scenario B maps naturally to an SFC: RU must be connected to a selected vDU and then to a vCU over packet fronthaul. VNFs (vDU/vCU) are embedded on substrate hosts (servers/switches), and compatibility plus fronthaul capacity/latency constraints govern the chain embedding [14].

2.2.2 Computational Complexity

VNF placement and SFC embedding are NP-hard in general. The orchestrator must decide host assignments for each VNF (e.g., vDU) and select routes between them; the joint placement–routing search space grows exponentially [15]. Even the RRH–BBU assignment in Scenario A becomes NP-hard once capacity/latency constraints are enforced: RRH act as items with heterogeneous loads, BBUss as bins with fixed capacity (Bin Packing), or as a Generalized Assignment variant. Hence exhaustive enumeration is infeasible beyond small instances, motivating exact-but-limited baselines and scalable heuristics/learning for larger cases.

2.3 A Review of Solution Paradigms

Given the NP-hard nature of VNF and SFC orchestration, the academic literature has investigated a diverse array of solution methodologies[16][15]. These approaches can be broadly classified into three primary paradigms: exact optimization through mathematical programming, scalable heuristics and metaheuristics, and adaptive learning-based methods. Each paradigm offers distinct trade-offs among solution optimality, computational scalability, and modeling flexibility.

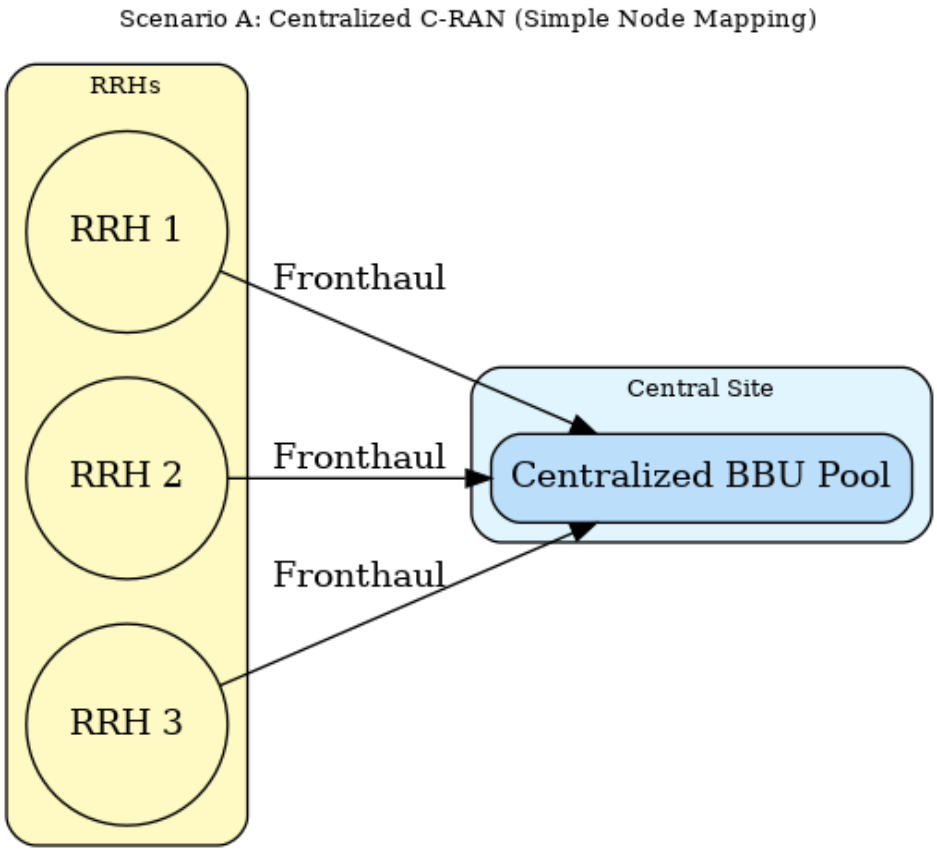


Figure 2.1: This figure illustrates Scenario A, in which a centralized baseband unit pool serves multiple RRHs over a constant-bit-rate CPRI fronthaul implementing functional split 8.

2.3.1 Exact Optimization via Mathematical Programming

The most rigorous approach to addressing combinatorial optimization problems is through mathematical programming, predominantly utilizing ILP or MILP[17].

Formulation and Purpose

This methodology entails formulating the placement and chaining problem as a formal optimization model. The model comprises an objective function that needs to be minimized (e.g., total operational cost) alongside a set of linear constraints. Typically, binary decision variables are employed to represent

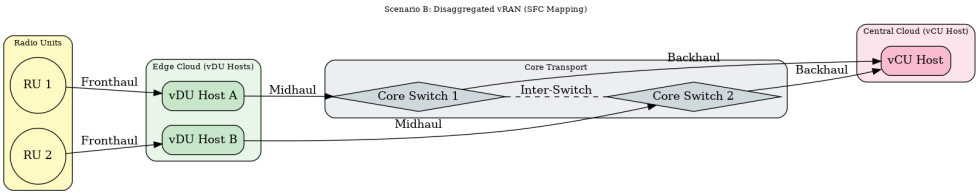


Figure 2.2: This figure illustrates Scenario B, an advanced disaggregated and virtualized RAN architecture in which the baseband unit is split into a virtualized distributed unit and a centralized unit, with functions deployed as software on general-purpose processors and radio units connected over packet-based fronthaul using eCPRI.

placement decisions; for instance, a variable x_{ij} takes the value of 1 if VNF i is placed on host j, and 0 otherwise. Constraints are then established to enforce rules such as resource capacity limits on hosts and fronthaul links.
A canonical example of this approach is the work by Bari et al. (2016), who defined the "VNF Orchestration Problem" (VNF-OP) and formulated it as an ILP[15]. They used the commercial solver IBM CPLEX to compute solutions that provably minimize the network's operational expenditure (OPEX) by optimally placing and routing VNFs.

Strengths and Weaknesses

The primary strength of the ILP/MILP paradigm lies in its guarantee of optimality. If the solver completes its execution, it certifies that the solution provided is indeed the global optimum. This characteristic renders ILP an invaluable scientific tool, offering a definitive benchmark against which the quality of other non-exact algorithms can be evaluated. For small-scale problems like our foundational Scenario A, an ILP solver could likely identify the provably best method for connecting RRH to the central BBU pool.
However, this approach's critical weakness resides in its lack of scalability. Solving an ILP constitutes an NP-hard problem; consequently, solvers' runtime escalates exponentially with increasing problem size. This limitation has been consistently documented in academic literature, with numerous studies indicating that ILP-based approaches become "practically intractable" as network size and complexity grow. In our Scenario B—characterized by multiple types of virtualized processing units, heterogeneous links, and intricate routing decisions—a pure ILP formulation would rapidly become computationally infeasible. Bari et al. explicitly state that their ILP model is only suitable for "small scale networks" and immediately propose a heuristic

algorithm to solve “larger instances”. Their experimental results showed that their heuristic could find solutions within 1.3 times of the optimal, a comparison that was only possible on the small topologies where the ILP could terminate in a reasonable time. This finding is echoed across the field, with numerous studies noting that ILP-based approaches become “practically intractable” as network size and complexity increase[15].
This has led to a near-ubiquitous research pattern: a paper will first propose a formal ILP model, demonstrate its correctness on small instances, show that it fails to scale, and then introduce a novel heuristic as the main contribution, using the ILP as a baseline for validation. This recurring pattern is more than just a standard paper-writing convention; it represents a collective acknowledgment by the research community that pure mathematical programming, while theoretically elegant, is fundamentally insufficient for solving real-world, large-scale orchestration problems. The community has effectively been proving the value of heuristics by comparing them against a benchmark that is known to be impractical. This methodological cycle highlights a clear need for alternative paradigms that can break this trade-off, offering better scalability than ILP without completely abandoning the formal guarantees and modeling rigor that heuristics lack.

2.3.2 Heuristics and Metaheuristics

To overcome scalability limits of exact solvers, heuristics and metaheuristics trade optimality guarantees for high-quality feasible solutions within practical time, even at large scale.

genetic algorithm

One widely recognized metaheuristic is the genetic algorithms (GAs), which draws inspiration from the principles of natural selection. Within the GA framework, a potential solution—such as a complete mapping of RU to DUs in Scenario B—is encoded as a ”chromosome.” An initial population consisting of randomly generated solutions is iteratively refined through genetic operators such as crossover and mutation. GAs have been successfully applied to optimization problems in C-RAN environments. In small-scale test cases, they can achieve performance that closely approximates the optimal solution obtained via ILP. More importantly, GAs remain effective in significantly larger network scenarios where ILP solvers become computationally infeasible. A delay-aware GA has also been proposed for SFC placement in 5G edge computing environments. The authors

demonstrated that their GA-based approach achieved performance comparable to that obtained through ILP on small-scale instances. Crucially, this method exhibited scalability, performing efficiently in large-scale network settings where conventional ILP solvers failed to generate feasible solutions within reasonable timeframes[18]. This study illustrates the core advantage of metaheuristics: achieving near-optimal results while substantially improving computational efficiency.

variable neighborhood search

Another powerful metaheuristic is variable neighborhood search (VNS). The fundamental concept of VNS involves systematically exploring various ”neighborhoods” of a current solution to evade local optima. A neighborhood refers to a set of solutions that can be attained through minor modifications, such as relocating a VNF to a different host. By methodically altering the type of move, VNS facilitates both an intensive local search and an extensive exploration of the solution space.
Luizelli et al. (2017) introduced an advanced ”fix-and-optimize” heuristic that integrates VNS for addressing the challenges associated with VNF placement and chaining[19]. Their algorithm has been shown to be highly effective and scalable, successfully identifying high-quality solutions in large-scale scenarios involving hundreds of VNFs—a problem size that significantly exceeds the capabilities of any exact ILP solver.

Beyond exact optimization techniques, clustering-based heuristics have been intensively explored as scalable strategies for Radio Access Network (RAN) resource orchestration. Among these heuristics, the K-Means algorithm has become a prevalently employed approach for preprocessing Remote Radio Head (RRH) assignment problems. By partitioning RRH into clusters prior to mapping them to BBUs or vDUs, the overall solution space is substantially diminished. This reduction enables more rapid and tractable orchestration in large-scale deployments[20].
Multiple studies have delved into geographical clustering, wherein RRH are categorized according to spatial proximity (such as latitude and longitude coordinates). This methodology has been demonstrated to efficiently address fronthaul latency limitations while simultaneously minimizing the occurrence frequency of inter-BBU handovers. This is because mobile users tend to move more frequently within clusters of neighboring RRH [21].
Beyond the static spatial features, researchers have put forward temporal and spatio-temporal clustering strategies. By making use of traffic profiles

obtained from call detail records (CDRs) or other usage traces, the K-Means algorithm can identify RRH with similar demand patterns. For example, it can distinguish those in business districts where the demand peaks during working hours from those in residential areas experiencing evening traffic surges. This kind of clustering facilitates proactive resource allocation, enabling the BBU or vDU pools to dynamically adjust their capacity in accordance with the predicted demand. To more effectively capture the structure of time-series data, the standard K-Means has been enhanced with distance metrics such as Dynamic Time Warping (DTW). This improvement enhances the clustering performance for traffic patterns that are temporally misaligned [22]. *Trade-off:* clustering reduces problem dimensionality but may increase inter-cluster routing or handover costs if cluster borders misalign with traffic flows; we account for this in evaluation by reporting both feasibility and cost metrics.

Limitations of Heuristic Approaches

Despite their scalability and practical utility, heuristic approaches exhibit significant limitations. Primarily, these methods do not guarantee global optimality; the solutions they yield may be suboptimal. Furthermore, quantifying the extent to which a solution deviates from the true optimum is often challenging without an ILP baseline, which is frequently unavailable for large instances.
More relevant to the objectives of this thesis, heuristic methods are fundamentally designed to converge on a single solution—the best one identified during their search process. They lack an intrinsic mechanism for systematically discovering and enumerating all optimal solutions or even multiple distinct high-quality alternatives. Although one might run a stochastic heuristic multiple times with different random seeds, this approach does not ensure that all optima will be found and may instead result in numerous duplicate or inferior solutions.

2.3.3 Learning-Based Approaches for Dynamic Environments

A more recent line of research has investigated the application of Machine Learning (ML), particularly RL (RL), in the domain of network resource allocation. The primary motivation behind these approaches is to effectively manage dynamic environments characterized by rapidly and unpredictably changing network conditions, such as variations in traffic demands[23].

RL for Online Orchestration

In a RL framework, an intelligent ”agent” learns a ”policy” for making placement decisions. The agent interacts with the network environment over numerous training episodes, receiving numerical ”rewards” for actions that yield favorable outcomes (e.g., low latency and high resource utilization) and incurring penalties for actions that result in unfavorable outcomes (e.g., constraint violations). Through trial and error, the agent develops a policy that maps network states to placement decisions designed to maximize its expected long-term reward.

Several studies have effectively applied Deep RL—which employs deep neural networks to approximate the policy—to address the VNF/SFC placement problem. A significant advantage of DRL lies in its operational speed; once trained (a process that can be time-consuming when conducted offline), the agent is capable of making placement decisions for new service requests almost instantaneously. This characteristic renders it particularly suitable for online scenarios where real-time decision-making is essential, especially in contexts where executing a full ILP or even employing complex heuristics for each request would prove too slow[24].

Limitations

While promising for dynamic environments, RL-based approaches exhibit several fundamental limitations similar to those of heuristics when analyzed through the lens of the Learning as a Service (LaaS) problem. First and foremost, these methods do not provide guarantees of optimality; the learned policy may converge to a suboptimal strategy. Second, enforcing hard constraints poses significant challenges. Constraints such as capacity limits or mutual exclusivity are typically incorporated as penalties within the reward function. Consequently, while the agent learns to avoid violating these constraints due to incurred penalties, this does not offer the same stringent assurance of satisfaction that explicit constraints in a formal model would provide.

Finally, and most critically for this study, the RL paradigm fundamentally focuses on identifying a single best action to take in any given state. It lacks the capability to enumerate multiple distinct and equally optimal solutions. The primary objective is to discover one effective policy rather than characterizing the entire landscape of optimal policies.

2.4 Research Gaps in RAN Orchestration

The preceding critical review of the state-of-the-art highlights a significant and largely unaddressed gap in the existing literature. Current resource orchestration methods—whether they are exact, heuristic, or based on learning—are inadequately suited to meet the specific demands of complex and heterogeneous environments such as vRAN model (Scenario B). This identified gap is characterized by two primary deficiencies: firstly, the inability to enumerate multiple optimal solutions; and secondly, the absence of a flexible framework for modeling intricate logical constraints.

2.4.1 Gap 1: Lack of Multi-Solution Enumeration

The vast majority of prior research is intentionally designed to yield a singular solution for any specific problem instance. An ILP solver, by default, concludes its operation upon identifying and validating the optimality of one solution. Heuristic and metaheuristic algorithms are specifically developed to converge towards the single best configuration they can discover. Likewise, a RL agent acquires a policy that prescribes one sequence of actions in response to a given request.

In numerous traditional network optimization scenarios, identifying a single optimal solution is sufficient. However, in complex orchestration systems, this becomes a major limitation. Providing the system operator or end-user with a comprehensive set of all optimal solutions offers significant practical advantages. First, operational flexibility is greatly enhanced. If a request can be satisfied by multiple distinct equipment combinations, the orchestrator is better positioned to adapt. For example, if a server hosting a vDU becomes unavailable due to maintenance, the system can seamlessly switch to an alternative configuration without re-solving the problem. Second, multiple optimal solutions enable user preference and choice. Users may favor equipment from specific vendors based on software familiarity or debugging needs. The availability of alternative configurations ensures these preferences can be respected without sacrificing optimality. Third, complete enumeration improves system resilience and supports strategic planning. A system with only one valid configuration is inherently fragile—any failure in a critical component can lead to failure. In contrast, systems with many interchangeable configurations are more robust and fault-tolerant. This knowledge is also valuable for capacity planning and identifying potential resource bottlenecks. Retrofitting existing methodologies for the purpose of performing solution

enumeration presents significant challenges and inefficiencies. Although it is possible to compel an ILP solver to identify multiple solutions by incrementally introducing "solution-cutting" constraints—essentially adding a constraint that excludes previously identified solutions—this iterative process proves cumbersome to implement and can greatly exacerbate the already considerable computational burden. Moreover, running a stochastic heuristic multiple times fails to guarantee the discovery of all optimal solutions or even distinct solutions in each iteration. Therefore, there is a pressing need for an approach that treats *solution enumeration* as a first-class objective. In this thesis we target (i) enumeration of *all optimal* solutions (same objective value), and (ii) optional *diversity* among returned solutions, measured via Hamming distance over device selections or path differences.

2.4.2 Gap 2: Inflexible Constraint Modeling

The second critical gap pertains to the expressiveness of modeling frameworks. The resource allocation problem described in Scenario B includes a variety of complex logical constraints that go far beyond the simplistic capacity limitations typically addressed in the VNF/SFC literature. In disaggregated radio access networks, several constraint types are particularly important. First, there are heterogeneous requirements: different types of radio units (RUs) may require fronthaul links with different capacities. Some RUs may need high-throughput fiber links, while others can tolerate lower bandwidth. Second, hardware-specific capabilities must be considered, such as switches with ports that support protocol conversion, which are not universally available. Third, mutual exclusivity arises when certain devices cannot coexist due to interference or software driver conflicts. Finally, there are dependency and implication constraints, such as when selecting a virtualized distributed unit (vDU) mandates allocating a cloud server with sufficient CPU resources. While it is feasible to represent such logical rules within an ILP model, this approach is often cumbersome and inefficient. The incorporation of each rule necessitates the introduction of additional binary variables alongside meticulously crafted linear inequalities, frequently employing "big M" formulations. For instance, a mutual exclusivity constraint between devices A and B—represented by binary variables x_A and x_B—would require a constraint such as $x_A + x_B \leq 1$. Similarly, a cardinality constraint like "select exactly one from A, B, C" would be expressed as $x_A + x_B + x_C = 1$. As the number of these rules escalates into the dozens or even hundreds, the ILP model may become unwieldy, challenging to maintain, and less efficient

for solvers to process.
Some prior research has integrated limited logical constraints. For example, Allybokus et al. (2018) explicitly incorporated anti-affinity rules in their ILP model to enhance service reliability by preventing certain VNF from being co-located on the same physical host. This work represents a significant advancement; however, it also underscores an existing problem: even a single type of logical rule introduces considerable complexity into the model. A Lab-as-a-Service system necessitates a framework that can naturally and efficiently articulate a diverse array of such constraints. Heuristic approaches are even less suitable in this context since they would demand custom ad-hoc logic tailored for each type of constraint—a solution that lacks scalability and maintainability.

2.5 Proposed Approach Based on Constraint Programming

To address the identified gaps in multi-solution enumeration and expressive constraint modeling, this thesis proposes the utilization of CP. Specifically, we leverage the CP-SAT solver from Google's OR-Tools suite, which is a state-of-the-art hybrid solver that integrates the strengths of CP, satisfiability (SAT) solving, and mathematical programming. The CP-SAT framework provides a powerful and elegant approach for modeling and resolving complex combinatorial problems.
CP is a declarative paradigm for discrete optimization. Users define variables, domains, and constraints; the solver combines propagation with systematic search to find assignments satisfying all constraints (and optimizing an objective). CP-SAT in OR-Tools hybridizes SAT, CP, and mixed integer programming (MIP) techniques for Boolean/integer models with linear and logical constraints.
Minimal mapping in our context:

```
variables  = { select RU/DU/CU instances (Bool),
               select links/paths (Bool) };

constraints = { ExactlyOne per category,
                AtMostOne for conflicts,
                Implications for dependencies,
                capacity on hosts/links };
```

```
objective   = minimize cost (devices/links/energy) or latency.
```

2.5.1 Expressiveness of Constraints

The primary advantage of CP for our problem lies in its rich and expressive modeling language. Unlike MILP, which is limited to linear constraints and often requires complex reformulations (e.g., big-M linearizations), constraint programming natively supports a wide range of logical and combinatorial constraints. This makes it particularly well-suited for modeling the complex orchestration rules in our LaaS environment and Scenario B.
These requirements can be concisely and accurately expressed using high-level primitives available in the CP-SAT solver provided by Google OR-Tools:

- `AddExactlyOne()` is used to model exclusivity, ensuring that exactly one device is selected from a category.
- `AddAtMostOne()` handles mutual exclusivity constraints among conflicting devices.
- `AddImplication()` expresses conditional dependencies between components.
- `AddBoolOr()` and `AddBoolAnd()` enable construction of more complex logical relationships.

This declarative modeling style offers several advantages. The CP model is often shorter, easier to maintain, and avoids the numerical instability and complexity introduced by linearization techniques commonly required in MILP. Prior studies have shown that CP solvers such as CP-SAT are especially effective in networking contexts involving logical dependencies, policy enforcement, or component compatibility constraints.

2.5.2 Native Support for Multiple Solutions

A second, crucial advantage of the CP paradigm is its inherent capacity to systematically enumerate multiple solutions. This feature directly addresses the first identified research gap. CP solvers function fundamentally as search engines that navigate the solution space. While they can be configured to halt upon discovering the first feasible (or optimal) solution, they can equally be directed to persist in searching for all other valid solutions.

The OR-Tools CP-SAT solver offers a straightforward application programming interface (API) designed specifically for this purpose. A solution callback can be implemented, which is triggered by the solver each time it identifies a new solution satisfying all constraints. In cases where an objective function exists, this callback can facilitate the collection of all solutions that share the same optimal objective value. This capability is native. To enumerate *optimal* solutions we use a two-phase procedure:

1. Solve the model once to obtain the optimal objective value z^*.
2. Add a constraint fixing the objective to z^*, and run `SearchForAllSolutions` with a `CpSolverSolutionCallback` to record distinct assignments (optionally enforcing diversity via Hamming-distance constraints or `AddAllowedAssignments` filters).

This avoids iterative "no-good" cuts and yields the complete set of optimal lab configurations.By capitalizing on this feature, our approach enables us to generate a comprehensive list of all optimal laboratory resource combinations for any given request, thereby providing both the Cloud-RAN Lab system and its users with enhanced flexibility and insights—advantages unattainable through single-solution methodologies.

2.5.3 State-of-the-Art Performance and Scalability

Modern hybrid solvers, such as CP-SAT, have significantly transformed the performance landscape in combinatorial optimization. By seamlessly integrating highly efficient SAT solving techniques with robust methods from the MILP domain—such as cutting planes—CP-SAT has demonstrated competitiveness with, and often superiority over, leading commercial MILP solvers across a diverse array of combinatorial optimization problems, especially those characterized by substantial logical structure.
These advances suggest that choosing CP-SAT for expressiveness and enumeration need not sacrifice performance. For problems dominated by logical structure—such as our RAN allocation—CP-SAT is often competitive with, and sometimes superior to, MILP-based approaches, while pure flow-heavy models may still favor MILP.

2.6 Synthesis and Comparative Analysis

This chapter has established the context for the thesis by linking the practical challenges associated with modern Radio Access Network (RAN)

architectures to foundational academic issues concerning VNF placement and SFC orchestration. A comprehensive review of prevailing solution paradigms—namely, exact optimization, heuristics, and RL—has uncovered two notable gaps: insufficient support for multi-solution enumeration and inadequate frameworks for modeling complex logical constraints present in heterogeneous, disaggregated environments.

The CP paradigm, particularly through the application of modern CP-SAT solvers, has been identified as a methodology uniquely equipped to address these shortcomings. Its declarative modeling language offers the necessary expressiveness to accurately represent intricate compatibility and hardware-specific rules directly. Additionally, its search-based approach provides inherent support for enumerating all optimal solutions, thereby granting essential flexibility to the orchestrator. Lastly, its state-of-the-art hybrid architecture ensures that these advantages are achieved without compromising performance.

To crystallize this analysis, Table 2.1 provides a comparative summary of the solution paradigms discussed, evaluating them across the key dimensions of optimality, scalability, constraint expressiveness, and multi-solution support. This comparison underscores the rationale for selecting a CP-SAT based approach to address the research problem at hand.

In light of this comparison, the CP-SAT approach emerges as the most promising method for achieving the dual objectives of optimality and flexibility inherent in the LaaS orchestration problem. It effectively addresses the gap left by previous methods by facilitating the efficient enumeration of all optimal solutions while accommodating the complex and heterogeneous constraint landscape associated with O-RAN laboratory resources.

The subsequent chapters will build upon this foundation. Chapter 3 will provide a formal definition of the LaaS resource allocation problem and present our comprehensive CP-SAT model. Chapter 4 will then detail its implementation, followed by Chapter 5, which will showcase experimental results demonstrating how this approach successfully enumerates optimal solutions under realistic laboratory constraints, thereby highlighting the advantages identified in this critical review.

Table 2.1: Comparison of solution paradigms across key dimensions.

Approach	Optimality Guarantee	Scalability	Constraint Expressiveness	Multi-Solution Support	Examples / Tools
ILP/MILP Optimization	Finds global optimum (single solution) if solved to completion.	Limited by exponential worst-case complexity; practical for small-to-medium instances only.	Linear constraints only; must linearize logical relations (e.g., using big-M formulations), which adds complexity and can weaken the model.	Not inherent. Solvers return one optimum by default. Obtaining multiple optima requires cumbersome iterative resolving with added constraints.	CPLEX, Gurobi. Widely used in NFV studies for finding optimal VNF placements in small networks.
Heuristics / Metaheuristics	No guarantee of optimality. Produces a feasible solution, which is often near-optimal but lacks a formal quality bound.	High scalability. Can handle large-scale problems in reasonable time, though solution quality may degrade as size increases.	Problem-specific handling. Can include custom logic for rules, but lacks a general, declarative language for constraints, making it hard to maintain and verify.	Typically one solution per run. Randomization can yield different solutions on multiple runs, but this is not a systematic enumeration of all optima.	GAs, VNS, Greedy Algorithms.
RL	No formal optimality guarantee. The trained agent aims to maximize expected long-term reward, which may correspond to a suboptimal policy.	Fast decisions at runtime (after training). Training phase can be time-consuming. Scales to large state spaces with deep neural networks.	Constraints must be encoded into the reward function (via penalties) or the policy structure. Cannot strictly enforce hard constraints, which may be violated during learning.	One solution per request (the agent's chosen action/policy). Does not enumerate multiple distinct solutions; focuses on learning a single "best" action.	DRL frameworks (e.g., TensorFlow, PyTorch). Applied to dynamic VNF placement in response to changing traffic.
Constraint Programming (CP-SAT)	Finds optimal solution(s). Can be configured to perform a complete search to find and enumerate all feasible or all optimal solutions.	High scalability with modern hybrid solvers. Can tackle large combinatorial problems, with performance competitive with or superior to MILP on many problem classes.	Very expressive. Natively supports a rich set of logical and combinatorial constraints (e.g., `AllDifferent`, `AtMostOne`, if-then) alongside standard linear constraints.	Native support for multiple solutions via the underlying search mechanism. The solver can be programmed to continue searching after finding one solution to enumerate all others.	Google OR-Tools CP-SAT, Choco, MiniZinc. Used for flexible SFC deployment with complex policies.

Chapter 3

Method

This chapter delineates the research methodology employed in this thesis, providing a comprehensive overview of how the orchestration problem within cloud RAN testbeds is systematically investigated and addressed. The methodological approach adheres to a structured pipeline: initially, we situate our work within a well-defined research paradigm, followed by an exposition of the experimental design that includes baseline comparisons and evaluation metrics.

Subsequently, we introduce the formal optimization model that encapsulates the fundamental resource allocation challenge, accompanied by algorithmic strategies devised to enhance scalability and practical applicability. Finally, we elaborate on the measures taken to ensure validity and reliability within this research context. This framework establishes a transparent and rigorous foundation for the ensuing implementation and results chapters.

3.1 Research Paradigm and Overall Approach

This thesis adopts a positivist and engineering-oriented research paradigm. The central aim is to develop, formalize, and evaluate algorithmic strategies for orchestrating cloud RAN testbeds. In this paradigm, knowledge is advanced by formulating the orchestration problem as a well-defined combinatorial optimization model, implementing solutions using state-of-the-art solvers, and empirically evaluating them under controlled experimental conditions.

The overall methodological flow is summarized in Figure 3.1. The

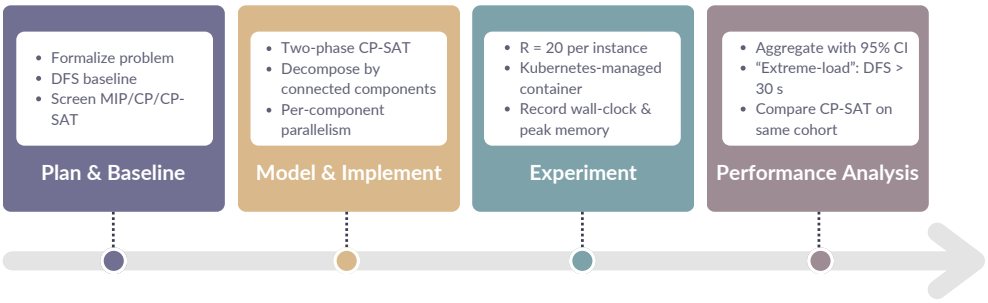


Figure 3.1: An overview of the research methodology, which combines formal modeling, algorithm design, and experimental validation in a lab-based Cloud-RAN testbed environment.

pipeline begins with a formal model of the orchestration problem, which serves as the theoretical foundation. Algorithmic strategies are then designed to solve the model efficiently, with a particular emphasis on scalability and the enumeration of multiple optimal solutions. To validate these strategies, a set of experiments is conducted in a lab testbed environment. These experiments follow a structured design: representative scenarios are selected, baseline methods are identified, and evaluation metrics are defined to ensure objective comparisons. The results are subsequently analyzed with respect to both optimality and computational performance.

By combining theoretical modeling with empirical validation, this paradigm ensures that the research maintains both scientific rigor and practical relevance. The methodology is thus not purely exploratory, nor purely mathematical; instead, it integrates formal optimization techniques with experimental evaluation in a realistic lab-as-a-service context.

3.2 Experimental Design

The experimental design defines how the proposed methodology is validated in a controlled yet realistic setting. The design aims to ensure that results are both reproducible and representative of the orchestration challenges faced in Cloud-RAN testbeds.

3.2.1 Experimental Environment

All experiments are conducted in a cloud-native LaaS platform that closely reflects the operational conditions of a cloud-RAN testbed. The platform is built on a Kubernetes-based container orchestration system, where

Table 3.1: Hardware configuration of the host machines (nodes) used in the testbed.

CPU (model/family)	Intel Xeon, 3.5 GHz
Physical cores / node	8
System memory	64 GB
OS (node)	Linux (x86_64)

Table 3.2: Kubernetes container resource quotas assigned to orchestration microservices.

CPU request / limit	0.25 cores / 2 cores
Memory request / limit	1 GiB / 1.5 GiB
Orchestrator	Kubernetes (Deployment/Pod)
Exposed port	8080/TCP

disaggregated RAN components are deployed as microservices. This setup allows fine-grained resource allocation and isolation, while maintaining the flexibility needed to emulate heterogeneous radio access environments.

Each host machine is provisioned with modern multi-core processors and ample memory to support parallel solver execution and large-scale orchestration scenarios. The detailed hardware configuration of the host nodes is provided in Table 3.1.

At the orchestration layer, all disaggregated RAN components and solvers are deployed as Kubernetes pods with controlled CPU and memory quotas. These quotas ensure fair allocation of resources across competing workloads and enable reproducibility of results across different runs. The container configuration is summarized in Table 3.2.

3.2.2 Scenarios and Workload Templates

Two representative scenarios are studied: (i) a baseline C-RAN model with centralized BBUs, and (ii) an advanced disaggregated vRAN model with vDU/vCU and heterogeneous RUs. Within each scenario, workload templates are generated to represent diverse orchestration requests. These templates vary in size (small to large) and complexity (homogeneous vs. heterogeneous device requirements), allowing us to evaluate both scalability and robustness of the proposed solution.

3.2.3 Baseline Methods

To ensure meaningful comparisons, the proposed CP-SAT-based approach is evaluated against three representative baselines drawn from distinct solver paradigms. First, exact optimization techniques such as integer linear programming (ILP) and mixed-integer linear programming (MILP) are used to obtain theoretically optimal solutions. However, their applicability is constrained to small-scale instances due to scalability limitations. Second, heuristic approaches including depth-first search (DFS) and clustering heuristics are considered. These methods can scale to larger problems and produce feasible solutions, but they do not guarantee optimality or completeness. Third, we include a constraint programming baseline using the Choco solver. This enables a fair comparison between the CP-SAT solver and another declarative solver, both of which support similar logical constraint representations but differ in underlying implementation and solver capabilities.

3.2.4 Evaluation Metrics

The evaluation focuses on two primary dimensions.

The first is solution quality, which is quantified by the number of additional switching devices required to fulfill a given configuration. This directly reflects the optimization objective defined in the problem formulation.

The second dimension is computational efficiency. It is assessed based on the runtime required to find the first optimal solution. For solvers that support solution enumeration, such as CP-SAT, we also measure the total time needed to enumerate all optimal configurations.

In addition to these core metrics, we also report secondary indicators where appropriate. These include memory usage during solving and the diversity of generated solutions, which is quantified using the average Hamming distance between valid configurations.

3.2.5 Reproducibility and Fairness

To ensure reproducibility, all experimental settings (solver versions, hardware resources, and template datasets) are documented, and the solver configurations are kept consistent across methods. Each experiment is repeated multiple times to account for stochasticity in heuristics, and average results with confidence intervals are reported. This design guarantees that performance

comparisons are fair and not biased by differences in implementation or environment.

3.2.6 Data Collection and Sampling

Instances are drawn from LaaS test templates and synthetic variants that mirror Cloud-RAN requests. No personal data are processed; the study concerns topology and solver performance only. We sample a diverse set of templates that vary in graph size, shape (motifs and substructures), and heterogeneity. In total, we evaluate 42 instances spanning small (3–5 RUs), medium (10–20 RUs), and large (30+ RUs) cohorts. Unless stated otherwise, each instance is run with $R = 20$ repetitions per solver configuration, and we report mean values with 95% confidence intervals.

3.2.7 Software Stack and Solver Configuration

Table 3.3 lists the software environment. All solvers are instantiated on equivalent models (same decision variables, constraints, and objective), and are restricted to the container resource quotas in Table 3.2. Threads are capped at 2.

Table 3.3: Software stack and solver versions.

Component	Version / Notes
Python	3.10.17
OR-Tools CP-SAT	9.14.6206
Gurobi (MIP)	11.0.2
Choco (CP)	4.10.14
NumPy / Pandas	Container-provided (see Appendix: env report)
Plotting	Matplotlib
Platform	Docker image on Kubernetes (cf. Table 3.2)

Baseline parity

- **CP-SAT**: A two-phase solving protocol is used: (i) minimize the total number of extra switching devices via the objective $\sum_{d \in D} y_d$, where y_d is a binary variable indicating whether device d is used;
  (ii) fix the optimal value and enumerate *all* optimal solutions using the `SearchForAllSolutions` API.
  The solver runs with component-level parallelism enabled via 2 threads.

- **Gurobi**: Configured with solution pool enabled: `Threads=2` for parallelism, `PoolSearchMode=2` to find multiple optima, a large `PoolSolutions` (e.g., 10^5), and `PoolGap=0.0` for strict optimality. Post-processing filters are applied to remove duplicates and suboptimal results.
- **Choco Solver**: A two-phase strategy similar to CP-SAT is adopted. After minimizing extra device usage, the `solve()` method is repeatedly called under a fixed cost to enumerate feasible optimal solutions.
- **DFS baseline**: A recursive backtracking algorithm is implemented with pruning strategies. Duplicate configurations are filtered in a post-processing step to ensure uniqueness.

3.2.8 Execution Protocol and Instrumentation

We adopt several mechanisms to ensure that evaluation results are precise, reproducible, and diagnosable.

Timing is measured using the `time.perf_counter()` function, capturing the full duration of solving, solution enumeration, and post-processing. Memory consumption is tracked using the `tracemalloc` module, which provides both the current and peak memory usage within the container. All logs and exceptions are systematically collected using Python's `logging` and `traceback` modules to support reproducibility and facilitate debugging. Each experimental run is subject to a fixed timeout threshold (typically 300 or 900 seconds). Runs that exceed this limit are considered censored; they are excluded from average and confidence interval calculations, but still contribute to success-rate metrics. For randomized algorithms, we assign distinct random seeds to each repetition. This ensures statistical diversity across runs while maintaining overall reproducibility.

Portability. Both host capabilities (Table 3.1) and container quotas (Table 3.2) are reported. This dual reporting ensures that reproductions can match the container envelope, even when executed on heterogeneous hardware platforms.

3.3 Formal Optimization Model

This section presents the formal optimization model that captures the orchestration problem in Cloud-RAN testbeds. The goal is to assign resources

and configure paths in a way that minimizes the number of additional switching devices while satisfying all functional and capacity constraints.

3.3.1 Sets and Parameters

- $\mathcal{R}$: set of radio units (RUs).
- $\mathcal{D}$: set of distributed units (Distributed Units (DUs)).
- $\mathcal{C}$: set of centralized units (Centralized Units (CUs)).
- $\mathcal{S}$: set of switching devices (including both existing and potential additional switches).
- $\mathcal{P}$: set of admissible paths connecting RUs, DUs, and CUs via switches.
- $cap(s)$: capacity of switch $s \in \mathcal{S}$ in terms of supported connections.
- $req(r)$: resource requirement of RU $r \in \mathcal{R}$.

3.3.2 Decision Variables

- $x_{r,d} \in \{0,1\}$: 1 if RU r is assigned to DU d, 0 otherwise.
- $y_{d,c} \in \{0,1\}$: 1 if DU d is connected to CU c, 0 otherwise.
- $z_p \in \{0,1\}$: 1 if path $p \in \mathcal{P}$ is activated, 0 otherwise.
- $w_s \in \{0,1\}$: 1 if switch $s \in \mathcal{S}$ is used, 0 otherwise.

3.3.3 Constraints

(C1) **Assignment constraint:** Each RU must be assigned to exactly one DU:

$$\sum_{d \in \mathcal{D}} x_{r,d} = 1, \quad \forall r \in \mathcal{R}.$$

(C2) **Chaining constraint:** Each selected DU must be connected to exactly one CU:

$$\sum_{c \in \mathcal{C}} y_{d,c} = 1, \quad \forall d \in \mathcal{D} \text{ with } \sum_{r} x_{r,d} \geq 1.$$

(C3) **Capacity constraint:** The load on each switch cannot exceed its capacity:

$$\sum_{p \ni s} z_p \cdot load(p) \leq cap(s), \quad \forall s \in \mathcal{S}.$$

(C4) **Switch activation constraint:** If a path uses switch s, then s must be activated:

$$z_p \leq w_s, \quad \forall p \in \mathcal{P}, \forall s \in p.$$

3.3.4 Objective Function

The optimization objective is to minimize the number of additional switching devices required:

$$\min \sum_{s \in \mathcal{S}} w_s,$$

where $\mathcal{S}$ includes candidate additional switches beyond the minimum required topology.

3.3.5 Discussion

While MIP is a robust and widely-adopted paradigm for linear optimization, its application to the disaggregated RAN orchestration problem presents significant challenges in terms of modeling expressiveness and solution completeness.

3.3.5.1 Linearization and Numerical Stability

The primary constraint in MIP is the requirement that all logical dependencies must be expressed as linear inequalities. In our Cloud-RAN orchestration problem, complex constraints such as mutual exclusivity between hardware components or conditional activations require explicit linearization. For instance, a simple logical implication $A \implies B$ (stating that if device A is selected, then device B must also be used) is typically formulated as:

$$B \geq A, \quad A, B \in \{0, 1\} \tag{3.1}$$

However, more complex requirements, such as cardinality constraints (e.g., "select exactly k resources if condition C is met"), necessitate the introduction of numerous auxiliary binary variables and large constants, known as "Big-M" formulations. This process not only increases the model's dimensionality but often results in a weaker Linear Programming relaxation. A weak relaxation

provides a poor lower bound for the branch-and-bound process, significantly increasing the computational effort required by solvers like Gurobi to prove optimality.

In contrast, CP-SAT natively supports logical primitives such as `AddExactlyOne`, `AddImplication`, and `AddBoolAnd`. By maintaining a compact and direct mapping from the problem domain to the solver, CP-SAT leverages Constraint Propagation and Conflict-Driven Clause Learning. These mechanisms allow the solver to prune infeasible regions of the search space based on logical conflicts rather than relying solely on numerical relaxation, making it more effective in logic-heavy network configuration scenarios.

3.3.5.2 Multi-Solution Enumeration Mechanisms

A critical research gap addressed by this thesis is the systematic enumeration of all optimal solutions. Standard MIP solvers generally find multiple solutions using a "Solution Pool," which collects feasible points encountered during the standard branch-and-bound search. However, this approach is often passive and does not guarantee a complete enumeration of all global optima unless the search parameters are set to be prohibitively exhaustive.

CP-SAT reframes solution enumeration as a first-class search objective. By utilizing a two-phase protocol—first identifying the optimal objective value Z^* and subsequently fixing it as a hard constraint—the solver uses the `SearchForAllSolutions` API to systematically traverse the entire optimal solution space. This architectural difference allows our framework to efficiently identify hundreds of interchangeable optimal configurations. In contrast, the alternative MIP strategy of using iterative "no-good cuts" to find subsequent solutions is computationally expensive and frequently becomes practically intractable as the solution space grows.

3.4 Algorithmic Strategy for Scalability

While the formal optimization model in Section 3.3 provides a rigorous abstraction of the orchestration problem, solving it directly at scale is computationally prohibitive due to its NP-hard nature. To address this, we design a set of algorithmic strategies that make the problem tractable in realistic testbed settings, while preserving completeness and optimality guarantees. The strategy integrates three key elements: problem decomposition, solver instantiation, and solution enumeration.

3.4.1 Problem Decomposition

The orchestration problem is first represented as a graph consisting of nodes (RUs, DUs, CUs, and switches) and edges (candidate links). To reduce the size of the search space, the graph is decomposed into independent or weakly connected subgraphs wherever possible. For instance, RUs located in distinct geographical clusters with no shared transport paths can be solved independently. This decomposition reduces problem size and allows parallel solving, significantly improving scalability without altering optimality.

3.4.2 Solver Instantiation

Each subproblem is then instantiated in the CP-SAT solver. Compared to MILP-based solvers such as CPLEX or Gurobi, CPSAT offers richer support for logical constraints and efficient propagation mechanisms. It also allows direct modeling of Cloud-RAN specific requirements such as mutual exclusivity of switches and dependency rules between DUs and CUs. To provide a fair comparison, equivalent models are also instantiated in an ILP solver and in an alternative CP framework (Choco). This ensures that performance differences can be attributed to solver characteristics rather than modeling bias.

3.4.3 Systematic Solution Enumeration

Once an optimal solution is found, the solver is reconfigured to enumerate all solutions with the same objective value. CPSAT natively supports this through solution callbacks, enabling the systematic exploration of the full optimal solution set. This is a crucial step in the LaaS testbed context, where multiple optima provide flexibility for user preferences, resilience against device failures, and insights into structural robustness of the underlying topology. By contrast, ILP solvers require iterative solution-cutting constraints to achieve enumeration, and heuristic methods cannot guarantee completeness.

3.4.4 Scalability Considerations

To ensure the approach remains scalable for large and complex testbed configurations, several strategies are employed. First, the overall problem is decomposed into smaller independent subproblems that can be solved concurrently across multiple CPU cores, enabling effective parallelization. Second, the search space is reduced through early pruning of infeasible

partial assignments. For example, if a tentative configuration would violate switch capacity constraints, it is discarded before further exploration. Finally, the solver supports incremental solving, which allows previously computed subproblem results to be reused when only minor modifications occur in the workload template. This significantly reduces redundant computation and improves responsiveness in dynamic orchestration scenarios.

3.4.5 Summary

This algorithmic strategy ensures that the proposed methodology remains computationally viable for medium- to large-scale orchestration scenarios. By combining decomposition, expressive solver instantiation, and systematic solution enumeration, the approach achieves both scalability and completeness, bridging the gap between theoretical optimality and practical feasibility.

3.5 Validity and Reliability

Ensuring the credibility of experimental results requires careful attention to validity and reliability. This section discusses the measures taken in the thesis to strengthen internal validity, external validity, construct validity, and reliability.

3.5.1 Internal Validity

Internal validity concerns whether the observed results can be confidently attributed to the chosen methodology rather than confounding factors. To control for this, all solvers (CP-SAT, MILP, classical CP, and DFS baseline) were instantiated on equivalent formal models of the orchestration problem, ensuring that observed differences arise from solver characteristics rather than modeling bias. Furthermore, all experiments were conducted in a containerized Kubernetes environment with fixed CPU and memory quotas. This eliminated uncontrolled variations in hardware performance and ensured consistent execution conditions.

3.5.2 External Validity

External validity concerns the extent to which results generalize beyond the testbed scenarios. While experiments were executed on representative lab templates inspired by realistic Cloud-RAN deployments, the scope remains

bounded to the LaaS context. The conclusions therefore generalize most strongly to research testbeds and operator labs, rather than live production environments. Nevertheless, the structural similarities of the orchestration problem to real-world RAN deployments suggest that the findings provide valuable insights for broader contexts, particularly for planning, simulation, and pre-deployment testing.

3.5.3 Construct Validity

Construct validity concerns whether the experimental evaluation accurately measures the theoretical constructs the study intends to investigate. In this work, the core objective is to assess the system's ability to systematically enumerate all optimal solutions under heterogeneous and complex constraints. To ensure alignment with this objective, three key measurement dimensions are defined.

First, completeness is evaluated by comparing the number of optimal solutions produced by the solver against the true set of optimal configurations, which is exhaustively known for small problem instances. Second, scalability is assessed by observing how runtime and memory usage evolve as the size and complexity of input templates increase. Finally, expressiveness reflects the solver's capacity to capture diverse and heterogeneous constraints without resorting to unnatural model transformations such as excessive linearization or handcrafted heuristics.

By designing evaluation metrics that directly map to the stated research goals—namely, multi-solution support, scalability, and modeling flexibility—the study ensures that the experiments provide valid evidence with respect to the intended constructs.

3.5.4 Reliability

Reliability concerns whether results can be reproduced by independent researchers. To strengthen reliability, each experiment was repeated $R = 20$ times, and confidence intervals at the 95% level were reported. The use of containerized execution with fixed resource quotas ensures reproducibility across different machines. Additionally, all solver scripts and orchestration templates have been version-controlled, and parameter settings (timeouts, solver flags, enumeration strategies) are explicitly documented. These measures enable others to replicate and extend the study with minimal ambiguity.

3.5.5 Summary

Through rigorous control of experimental conditions, alignment of metrics with research goals, and transparent documentation, this thesis ensures that results are both valid and reliable. While external generalizability to live production networks remains a limitation, the methodology and findings are robust within the targeted domain of LaaS-based Cloud-RAN orchestration.

3.6 Summary of Methodology

In summary, this chapter has established the methodological foundation of the thesis. We began by situating the study within a positivist and engineering-oriented research paradigm, and then detailed the experimental design used to ensure fairness and reproducibility. The orchestration problem was formally modeled as a combinatorial optimization problem, and algorithmic strategies such as graph decomposition and CP-SAT solver instantiation were introduced to address scalability challenges. Furthermore, validity and reliability considerations were explicitly incorporated to strengthen the rigor of the study. This comprehensive methodology sets the stage for the next chapter, where we present the implementation details and experimental evaluation of the proposed approach.

Chapter 4

Design and Implementation

This chapter presents the design and implementation of our CP-SAT–based solution for enumerating all valid laboratory resource combinations under constraints. Compared with the methodological formalization in Chapter 3, this chapter focuses on the software architecture, model-to-code mapping, core algorithms (two-phase CP-SAT protocol, graph decomposition, deduplication and merging), and the engineering decisions that ensure scalability, correctness, and reproducibility.

The chapter is organized as follows. Section 4.1 introduces the system architecture and core algorithms. Section 4.2 discusses correctness, equivalence, and complexity considerations. Section 4.3 specifies solver configurations and reproducibility measures. Section 4.4 describes robustness and failure handling. Section **??** summarizes how the design meets our research goals.

4.1 System Architecture and Core Algorithms

4.1.1 Architecture Overview

The proposed microservice is implemented using a modular architecture, with each component responsible for a distinct stage in the orchestration workflow. As shown in Figure 4.1, the system begins with an API layer that accepts JSON-based requests, validates the input topology, and orchestrates the end-to-end processing pipeline. This pipeline includes decomposition, solving, and result merging.

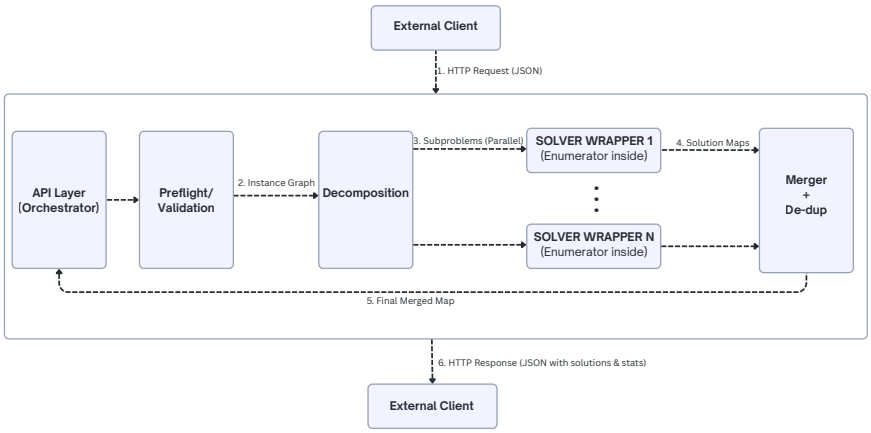


Figure 4.1: Modular architecture of the microservice and orchestration workflow.

The decomposition module analyzes the input graph and identifies independent connected components, which are then formulated as separate subproblems. Each subproblem is passed to the solver wrapper, which transforms the domain-level topology and resource constraints into a formal constraint model using the CP-SAT solver. The solving process follows a two-phase protocol: first minimizing the number of additional devices, and then enumerating all optimal configurations.

During the enumeration phase, a dedicated solution collector ensures that only unique configurations are retained, avoiding duplicates through online filtering. Once all subproblems are solved, the merger component incrementally combines the solution sets into a globally deduplicated result set that is returned to the user in JSON format.

4.1.2 Model-to-code mapping

The declarative model in Section 3.3 is implemented directly with OR-Tools CP-SAT:

- (C1) one group per node → `AddExactlyOne`.
- (C2) resource capacity → `Add(sum(...) <= S_r)`.
- (C3) mutual-domain compatibility → `AddBoolOr([x.Not(), y.Not()])`.
- (C4) extra-device activation → `AddMaxEquality`.

The objective $\min \sum_d y_d$ is set in Phase 1, then replaced by a fixed-cost equality for Phase 2 enumeration.

4.1.3 Two-phase optimal enumeration

The solution process is structured in two distinct phases to ensure both optimality and completeness. In the first phase, the solver minimizes the total number of additional switching devices, denoted as $\sum_d y_d$, in order to compute the optimal value $Z^\star$. Once this minimum value is identified, the model is modified for the second phase. Specifically, an additional constraint is introduced to enforce $\sum_d y_d = Z^\star$, thereby restricting the search space to only optimal solutions. The objective function is then cleared, and the solver invokes its multi-solution enumeration procedure using the `SearchForAllSolutions` method. This two-phase approach guarantees that all solutions returned in the second phase are globally optimal with respect to the original objective.

Enumeration is captured via a callback that computes a canonical *solution signature* (see Section 4.2) and discards duplicates on the fly.

4.1.4 Component decomposition and parallel solving

Each connected component is solved independently, leveraging structural independence. Pseudocode:

```
def solve_instance(instance):
    comps = get_nodes_subsets(instance)
    comp_inputs = [restrict(instance, nodes=c) for c in comps]
    with ThreadPoolExecutor(max_workers=2) as ex:
        comp_maps = list(ex.map(solve_component, comp_inputs))
    if any(len(m) == 0 for m in comp_maps):
        return []  # infeasible
    comp_maps.sort(key=len)
    merged = comp_maps[0]
    for m in comp_maps[1:]:
        merged = merge_two_solution_maps(merged, m)
    return format_results(merged)
```

The decomposition strategy leverages the inherent modularity of the problem instances. Each component is treated as a fully independent subproblem. The two-phase optimal enumeration protocol is executed on each component in parallel to find its local optimal solutions. The final global solution set is then derived from the Cartesian product of these local solution sets. This approach is valid under the assumption that both the objective function and all constraints are decomposable by component, as detailed in Section 4.2.

4.1.5 Deduplication and Merging of Solution Sets

Each component outputs a solution map keyed by a canonical signature. Merging is performed from smaller to larger maps to delay combinatorial blow-ups. Duplicate signatures are discarded in constant time.

4.2 Correctness and Complexity

This chapter establishes the correctness of the decomposition strategy used in the proposed orchestration system and analyzes the computational complexity of the overall approach. We first explain why solving each connected component independently preserves global optimality, and then discuss the computational implications.

4.2.1 Correctness of Decomposition

The divide-and-conquer strategy relies on the assumption that the orchestration problem can be separated into independent subproblems. The correctness of this separation follows from two key conditions that ensure decomposability.

4.2.1.1 Decomposable constraints.

All constraints in the orchestration problem, including resource capacities and compatibility rules, must apply only within individual connected components. No constraint may involve variables belonging to two different components.

4.2.1.2 Decomposable objective.

The global objective function must be additively separable across components. In the case of minimizing the number of additional switching devices, the global objective

$$Z = \sum_{d \in D} w_d y_d$$

can be written as a sum of per-component objectives

$$Z = \sum_{c \in C_{\text{comp}}} Z_c,$$

where each Z_c counts only the devices associated with component c.

4.2.1.3 Implication.

When both conditions hold, the entire optimization problem factorizes into independent subproblems. Let $\mathcal{S}_c$ denote the set of optimal solutions for component c. Since there are no inter-component constraints, any combination of local solutions $(s_1, s_2, \ldots, s_k)$ with $s_c \in \mathcal{S}_c$ forms a valid global solution. Moreover, because the global objective is additive, a global solution is optimal if and only if each selected s_c is locally optimal. Therefore, the set of globally optimal solutions is precisely the Cartesian product of the locally optimal solution sets.

4.2.2 Equivalence under deduplication

We define an equivalence relation on solutions: two assignments are equivalent if and only if each node is assigned the same candidate group and the same activated extra devices. Deduplication uses a stable hash of the tuple $(n, g_id, \text{resource_ids})_{n \in N}$ as signature. Deduplication ensures that, for each equivalence class of solutions, exactly one representative is retained while all other duplicate solutions are discarded. This guarantees both correctness and completeness of the enumeration process.

Remark. If a coarser equivalence (e.g., by resource family) is desired, the relation must be explicitly redefined and reported as such.

4.2.3 Computational Complexity and Resource Usage

The proposed formulation introduces $O(|N| + |G| + |D|)$ decision variables, where N denotes the set of nodes, G the candidate groups, and D the domain connections. Compatibility constraints across links contribute an additional $O(|L|)$ clauses. Although the first phase of CP-SAT optimization is NP-hard in the worst case, strong constraint propagation ensures that practical runtimes remain tractable for all evaluated instances.

The merging stage can, in theory, yield up to $\prod_{i=1}^{k} m_i$ valid configurations, where m_i is the number of optimal solutions in component i. To mitigate the potential memory overhead of such combinatorial growth, solution sets are merged incrementally, starting with smaller components. This ordering reduces intermediate peak memory usage and improves overall runtime stability.

Table 4.1: Key solver parameters and environment settings used across all experiments.

Parameter	Value
Time limit	300 s (wall-clock)
Parallel workers	2 (bounded by container quota)
CP-SAT workers	2 (`num_workers`)
Enumeration protocol	Two-phase with `SearchForAllSolutions`
Random seed	Fixed per run (deterministic)

4.3 Configuration and Reproducibility

This section outlines the configuration settings used throughout the experimental evaluation and describes the measures taken to ensure reproducibility. All experiments are executed within the same containerized environment, subject to fixed CPU and memory quotas. Random seeds, solver thread limits, and canonical ordering of solution signatures are used to guarantee deterministic behavior across runs.

4.3.1 Solver Configuration

The principal solver parameters are summarized in Table 4.1. These settings apply to all experiments unless otherwise noted, and they ensure that the solving conditions remain consistent across different inputs.

4.3.2 API Schema and Example

The microservice exposes a JSON-based API for submitting testbed topologies and retrieving the corresponding set of optimal resource combinations. Identifiers in the examples below are anonymized for clarity. The request encapsulates a list of candidate nodes and their associated constraints.

4.3.2.0.1 Example Request.

```
{ "data": { "testChannelCandidates": { "nodeCandidates": [ ... ] } } }
```

4.3.2.0.2 Example Response. The service returns a list of valid combinations and summary statistics describing the solving process:

```
{
  "validCombinations": [
    {
      "extraDeviceCount": 0,
      "nodeCandidates": [
        { "groupName": "NODE_B_0", "nodeName": "NODE_B",
          "resourceName": "RGW_DEV_01", "additionalResourceNames": [] },
        { "groupName": "NODE_A_0", "nodeName": "NODE_A",
          "resourceName": "BB_DEV_01", "additionalResourceNames": [] }
      ]
    }
  ],
  "statistics": {
    "graph_components": 1,
    "unique_solutions_found": 1,
    "solve_time_seconds": 0.06,
    "peak_memory_mib": 45.7,
    "version": "1.0",
    "seed": 42
  }
}
```


The field `additionalResourceNames` lists substitute resources within the same candidate group when such alternatives exist.

4.4 Robustness and Failure Handling

The system incorporates a comprehensive set of mechanisms to ensure robustness under a variety of load and input conditions.

4.4.1 Timeout Handling

Each solver run is subject to a fixed time limit of 300 s. When a run exceeds this limit, the solver is terminated and a standardized timeout code (`E_TIMEOUT`) is returned. Partial solutions are not provided, as doing so would violate the guarantee that all reported solutions are globally optimal.

4.4.2 Input Validation

All submitted instances are validated for structural correctness prior to solving. This includes checks for graph connectivity, candidate-group existence, and domain compatibility. Invalid requests result in explicit error codes such as

E_PARSE (for malformed JSON) or E_INVAL (for inconsistent or incomplete topology definitions).

4.4.3 Standardized Error Codes

To support reliable client-side handling, the API reports standardized error codes, including but not limited to:

- E_TIMEOUT: time limit exceeded,
- E_PARSE: invalid JSON input,
- E_INVAL: invalid topology or domain configuration,
- E_INFEAS: no feasible solution exists.

Chapter 5

Results and Analysis

In this chapter, we present a comprehensive empirical evaluation of the proposed CP-SAT-based optimization method in comparison to the baseline DFS algorithm. The objective of this analysis is to rigorously quantify the advantages associated with adopting a modern CP paradigm over a traditional imperative search technique for addressing complex resource combination problems. The evaluation is structured across four critical dimensions: correctness, which ensures the validity and completeness of the generated solutions; runtime performance, which measures the computational time required to identify solutions; memory efficiency, which assesses peak memory consumption during execution; and scalability, which examines each method's performance as problem size and complexity increase.
To facilitate a nuanced comparison, we employ a suite of metrics that includes direct runtime speedup, percentage of time saved, and memory savings. Additionally, a phase-wise breakdown of runtime is conducted to pinpoint specific computational stages where each algorithm incurs significant costs. Furthermore, we isolate and analyze a set of "extreme cases"—instances in which the baseline DFS algorithm's runtime exceeds 30 seconds—to underscore performance differentials under computationally challenging scenarios. The collective results from this multi-faceted analysis will demonstrate the superior robustness, efficiency, and scalability inherent in the CP-SAT approach.

5.1 Correctness of Results

A core requirement for any optimization approach is its ability to produce correct and complete sets of feasible solutions. Experimental comparisons

between the DFS baseline and CP-SAT have revealed significant deficiencies in the DFS implementation, particularly in scenarios involving cyclic graph topologies and shared resource constraints.

One notable limitation emerges in the presence of ring-like topologies. When the underlying graph includes cycles or wrap-around connections, a depth-first search strategy may encounter two distinct failure modes. Without proper tracking of visited nodes, the search can enter infinite recursion due to repeated traversal of cyclic paths. On the other hand, introducing naive cycle-avoidance logic—such as forbidding revisits—can cause premature pruning of legitimate configurations. These issues lead to overcounting redundant paths while simultaneously omitting valid cyclic combinations. As shown in Table 5.1, the DFS method reports 380 solutions for the ring-based ”Example A”, whereas the CP-SAT solver identifies only four, all of which are unique and valid. The discrepancy indicates both overgeneration and undercoverage in the DFS enumeration.

A second class of failure arises in configurations where resources are shared among multiple candidate groups. DFS requires precise state management to backtrack correctly when a shared resource is temporarily allocated along one branch of the search tree. If a resource is not correctly released during backtracking, subsequent branches falsely perceive it as unavailable. This can result in the exclusion of valid combinations. In ”Example B” from Table 5.1, DFS returns 2970 configurations, while CP-SAT identifies 3080. The missing 110 solutions highlight the impact of incorrect state recovery in DFS-based implementations.

These limitations are inherent to procedural enumeration strategies, particularly when complex constraints or shared dependencies are involved. In contrast, CP-SAT follows a declarative modeling paradigm. Instead of explicitly managing control flow and recursion, the problem is encoded as a set of variables and constraints, which are then interpreted by the solver.

This modeling paradigm offers two primary advantages. First, global constraints, such as those involving cycles or shared capacities, are natively supported and mathematically well-defined. Constructs like `AddBoolOr` or `AddCumulative` allow for expressive and correct formulations without the risk of implementation bugs. Second, the search strategy employed by CP-SAT is systematic and complete. Techniques such as branch-and-bound and conflict-driven clause learning enable the solver to exhaustively explore the feasible space and prove the optimality of the returned solutions.

In summary, CP-SAT provides a correctness guarantee that is absent in DFS-based enumeration. Its ability to model complex resource constraints

Table 5.1: Comparison of solution counts between DFS and CP-SAT

Instance	DFS solutions	CP-SAT solutions
Example A (ring topology)	380	4
Example B (shared resources)	2970	3080

and to systematically traverse the solution space makes it the preferred choice for problems requiring both completeness and reliability.

5.2 Runtime Performance

This section evaluates the runtime performance of the compared solving approaches under varying problem sizes and solution-space complexities. The analysis highlights how the CP-SAT solver achieves superior scalability and efficiency compared to a baseline DFS implementation.

5.2.1 Overall Scaling

Figure 5.1 presents a clear visual depiction of the scalability of each algorithm's runtime in relation to the size of the problem input, specifically the number of nodes.

- **The Exponential Bottleneck of DFS:** The DFS curve (represented by the orange line) demonstrates a pronounced upward trajectory, indicative of significant quadratic or even exponential growth. As the number of nodes increases from 10 to 14, the runtime escalates dramatically from approximately 10 seconds to nearly 50 seconds. This stark contrast highlights the brute-force nature inherent in its search methodology. In the absence of intelligent pruning mechanisms, each additional node can exponentially increase the size of the search space, resulting in what is commonly referred to as "combinatorial explosion." Furthermore, the algorithm's inability to learn from previous failures leads it to repeatedly traverse similar dead-end paths across different branches—a phenomenon known as "thrashing," which serves as a fundamental contributor to its suboptimal performance.
- **Near-Linear Scalability of CP-SAT:** In contrast, the performance curves of CP-SAT (single-threaded in blue and multi-threaded in

green) exhibit a significantly flatter profile, indicating remarkable near-linear scalability. Even when utilizing 14 nodes, the runtime remains approximately 5 seconds. The underlying power behind this performance is attributed to its advanced solving techniques. Constraint Propagation operates at each step of the search process to reduce the domains of future variables based on current decisions, effectively pruning extensive sections of the search tree before their exploration occurs. Conflict-Driven Clause Learning (CDCL) enables the solver to learn from previous errors. Upon encountering a conflict, it generates a new constraint (referred to as a ”clause”) that prevents similar conflicts from reoccurring anywhere within the search space. The synergy among these mechanisms empowers CP-SAT to navigate intelligently through the search space rather than traversing it blindly.

- **The Advantages of Multi-Threading:** Furthermore, the performance of the multi-threaded CP-SAT (represented by the green line) consistently outperforms that of its single-threaded counterpart (depicted by the blue line). This observation substantiates the efficacy of the CP-SAT architecture in harnessing modern multi-core processors through parallel execution of a diverse array of search strategies. When one thread identifies a solution or uncovers critical information—such as a strong new clause—it can be communicated to other threads, thereby enhancing the overall efficiency of the solving process.

5.2.2 Performance versus. Solution Space Size

While the input size, specifically the number of nodes, serves as a primary determinant of computational complexity, the density of the solution space—defined by the number of feasible solutions—provides a more nuanced understanding of the fundamental differences between DFS and CP using SAT solvers (CP-SAT). Figure 5.2 and Table 5.2 illustrate how their runtimes evolve as the count of solutions increases.

The red curve in Figure 5.2, representing DFS, exhibits a steep growth pattern that closely aligns with a power-law relationship between runtime and solution count, characterized by an exponent approaching 1. This behavior reflects the enumeration-based nature of DFS; to identify all possible solutions, it must systematically explore every potential path within the search tree. Consequently, its workload scales nearly linearly with the number of valid solutions. For instance, when the solution count escalates from 39 to 1,092—a

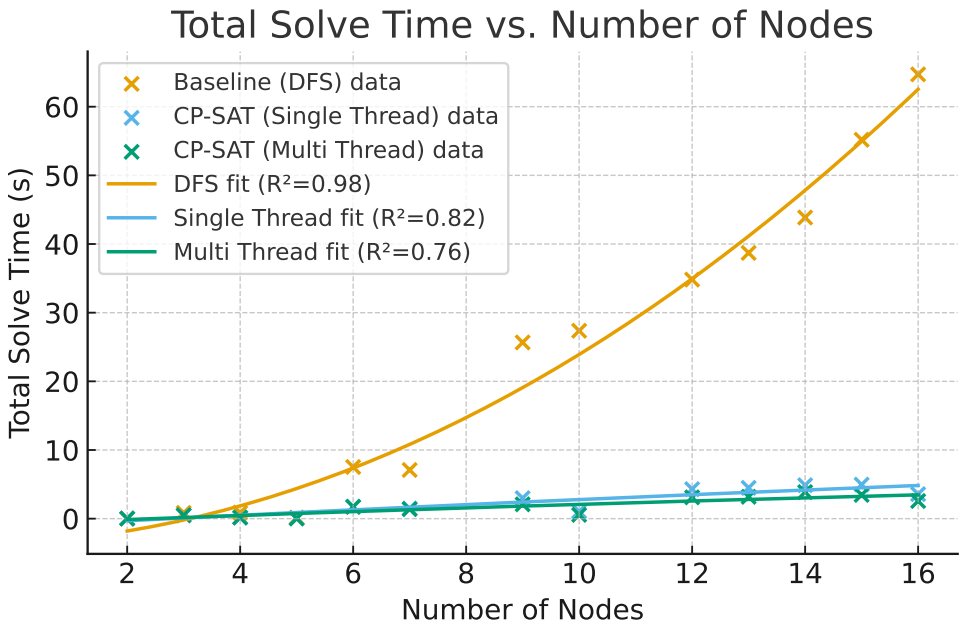


Figure 5.1: Runtime scaling with number of nodes (linear scale).

28-fold increase—the runtime rises from 2.18 seconds to 34.83 seconds. This observation underscores the significant enumeration burden placed on DFS and highlights its limited scalability.

In stark contrast, the blue curve representing CP-SAT exhibits a significantly weaker, sub-linear correlation, with a fitted exponent of approximately 0.7. This characteristic is indicative of a constraint-driven solver. Its primary function involves pruning infeasible branches within the search tree rather than enumerating feasible ones. Consequently, its runtime is more closely associated with the structure of the problem's constraints than with the sheer number of solutions available. Even as the solution count increases dramatically, the solver's robust propagation and pruning mechanisms effectively contain the search effort, resulting in only an approximate 1.7-fold increase in runtime over this range. This demonstrates a remarkable resilience to variations in solution space density—a quality that DFS lacks.

5.2.3 Quantifying the Performance Gap Through Speedup Analysis

Figure 5.3 quantifies the relative advantage of CP-SAT over the DFS baseline using the speedup metric (DFS runtime / CP-SAT runtime). The horizontal

Table 5.2: Key data points for runtime versus number of solutions.

Num of Solutions	DFS Runtime (s)	CP-SAT Runtime (s)	Speedup
4	0.014	0.028	0.51x
39	2.18	0.50	4.38x
1092	34.828	4.302	8.09x
1884	63.86	3.88	16.54x

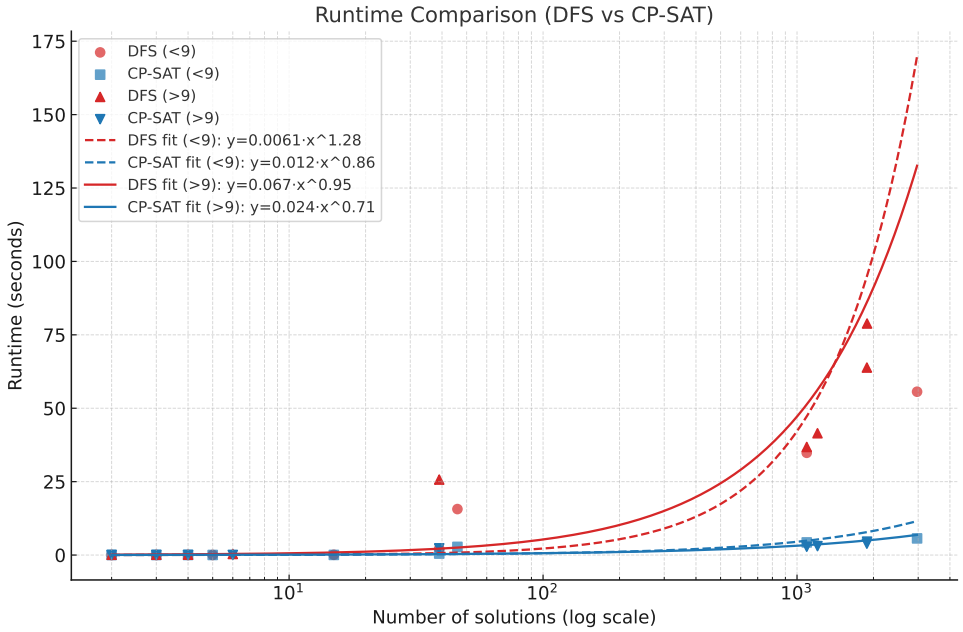


Figure 5.2: Runtime scaling with number of solutions (log-Linear scale).

axis represents the number of optimal solutions on a logarithmic scale, which serves as a proxy for *solution-space complexity*. Each point corresponds to one template instance, with separate cohorts distinguished by template size: (i) templates with ≤ 9 nodes, and (ii) templates with > 9 nodes.

For small templates (≤ 9 nodes), the fitted curve follows $y = 0.52 \cdot x^{0.42}$. The exponent (0.42) indicates a relatively steep growth trend as the number of optimal solutions increases, but the low constant factor (0.52) means that the absolute speedup remains limited. In this regime, CP-SAT's fixed modeling overhead—constraint construction and solver initialization—can offset its pruning advantage, resulting in only modest gains when the solution space is small.

For larger templates (> 9 nodes), the fitted curve is $y = 2.8 \cdot x^{0.24}$. Here

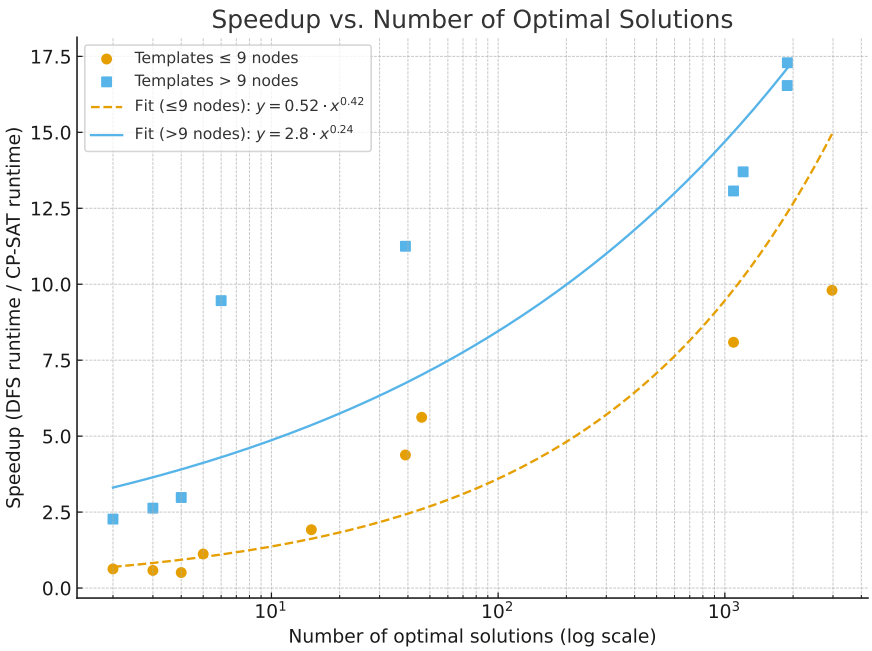


Figure 5.3: Speedup (DFS runtime / CP-SAT runtime) as a function of the number of optimal solutions (log-linear scale)

the exponent is smaller, reflecting a slower marginal growth rate, yet the higher constant factor (2.8 vs. 0.52) produces a much wider absolute gap. As a result, CP-SAT routinely achieves speedups greater than 10× and peaks near 17×. This reflects the rapidly escalating cost of exhaustive DFS enumeration in complex solution spaces, whereas CP-SAT's propagation-based search is able to prune large portions of the space even as the number of optimal solutions increases.

Overall, the results highlight two consistent insights. First, when the solution space is shallow, CP-SAT's relative advantage is reduced by fixed overheads. Second, as the number of optimal solutions grows—i.e., as solution-space complexity increases—CP-SAT's scalability advantage becomes increasingly evident. The divergence between the two cohorts underscores that while small templates do not fully exploit CP-SAT's strengths, larger and more complex instances reveal its substantial superiority over procedural enumeration.

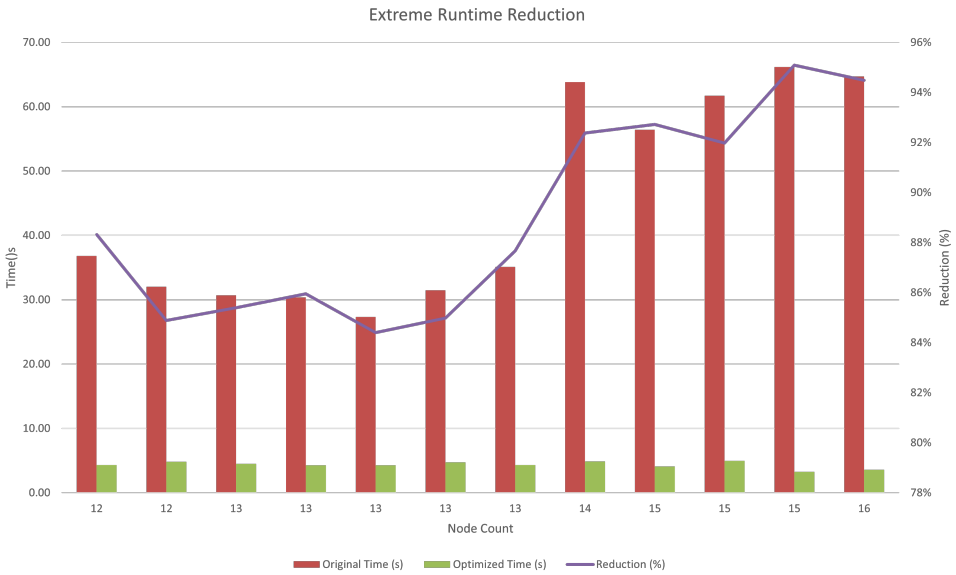


Figure 5.4: Extreme cases: DFS vs CP-SAT runtimes with percentage time saving.

5.2.4 Tractability Analysis Under Extreme Conditions

The practical implications of this scaling advantage are most vividly illustrated at the threshold of usability, as evidenced by the extreme cases presented in Figure 5.4. These instances, where DFS exceeds a runtime of 30 seconds, signify problems that are becoming computationally intractable under a naive approach.

The towering bars representing DFS, which extend beyond 60 seconds, starkly contrast with the nearly negligible bars for CP-SAT, which successfully solve identical problems in well under 5 seconds. The resulting reductions in runtime—ranging from 88% to over 94%—are transformative. This is not merely a marginal enhancement; it represents a critical distinction between a practical tool and one that remains theoretical. It converts a problem that could require up to a minute of computation—a considerable workflow bottleneck—into one that can be resolved almost instantaneously. This analysis demonstrates that while DFS encounters significant challenges as solution spaces expand, CP-SAT continues to provide fast and reliable performance, thereby positioning itself as a superior method for addressing computationally demanding large-scale problems.

5.3 Memory Efficiency

A comparison of peak memory usage, illustrated in Figure 5.5, underscores another significant advantage of the CP-SAT approach. While both algorithms exhibit similar memory footprints on smaller instances, their requirements diverge markedly as the graph size increases.

The DFS algorithm, implemented recursively, can consume hundreds of megabytes of memory for large graphs. This high consumption is primarily attributable to the depth of the recursion stack. In the worst-case scenario, the space complexity of DFS correlates with the longest path in the graph, which can be considerable. Each recursive call adds a frame to the call stack; thus, for deep and complex search trees, this results in substantial memory usage.

In contrast, CP-SAT maintains consistently low peak memory usage—typically remaining below tens of megabytes—even for its largest instances. This efficiency arises from its sophisticated internal architecture that employs highly optimized data structures, including lazy data structures designed to manage extensive numbers of learned clauses effectively. More importantly, its advanced pruning techniques ensure that the effective search tree explored by CP-SAT is considerably smaller and shallower than that traversed by DFS; consequently, this leads to fundamentally lower memory requirements. As demonstrated in Figure 5.5, across 20 experimental runs CP-SAT achieves average memory savings ranging from 40% to 70%, with relative advantages increasing alongside problem size.

This evidence substantiates that CP-SAT's superiority extends beyond runtime efficiency into aspects of memory scalability, rendering it a more practical solver for large-scale and memory-intensive problems.

5.4 Scalability and Decomposition

Figure 5.6 illustrates the time savings achieved by decomposing the original graph into k *independent* subgraphs, with the baseline being no decomposition (k=1). Each data point represents the sample mean accompanied by a 95% confidence interval derived from n repeated runs.

The trend observed is both monotonic and significant: at k=1, the mean time saving is approximately -20%, reflecting the fixed overhead associated with the decomposition framework when no partitioning occurs. This improves to 25% at k=2, 68% at k=3, and reaches 81% at k=4. The relatively narrow confidence bands suggest that these improvements are statistically robust.

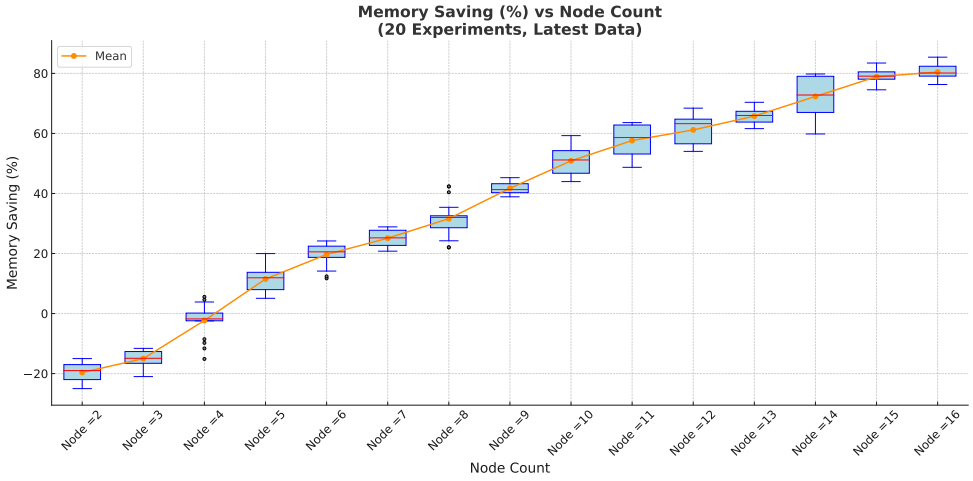


Figure 5.5: Boxplot of memory savings compared to DFS.

Although gains increase with higher values of k, it is noteworthy that the incremental benefit from transitioning from k=3 to k=4 is less pronounced than that observed between k=2 and k=3. This indicates diminishing returns likely attributable to partitioning overheads and occasional load imbalances among subproblems.

5.4.1 Why Decomposition is Beneficial

The independence of subgraphs allows the original search space to factor into a Cartesian product of smaller spaces. Consequently, solving k independent models effectively reduces branching at each decision level and facilitates parallel execution across subgraphs, resulting in significant savings in wall-clock time.

5.4.2 CPSAT versus DFS in the Context of Independence

Both methods leverage independence; however, CP-SAT exploits it more intensively. For each subproblem, CP-SAT employs constraint propagation and conflict learning to aggressively prune the local search space, often leading to superlinear reductions in runtime as models decrease in size. In contrast, DFS still engages in exhaustive enumeration within each subproblem; without cross-subgraph pruning, the overall runtime remains close to the cumulative

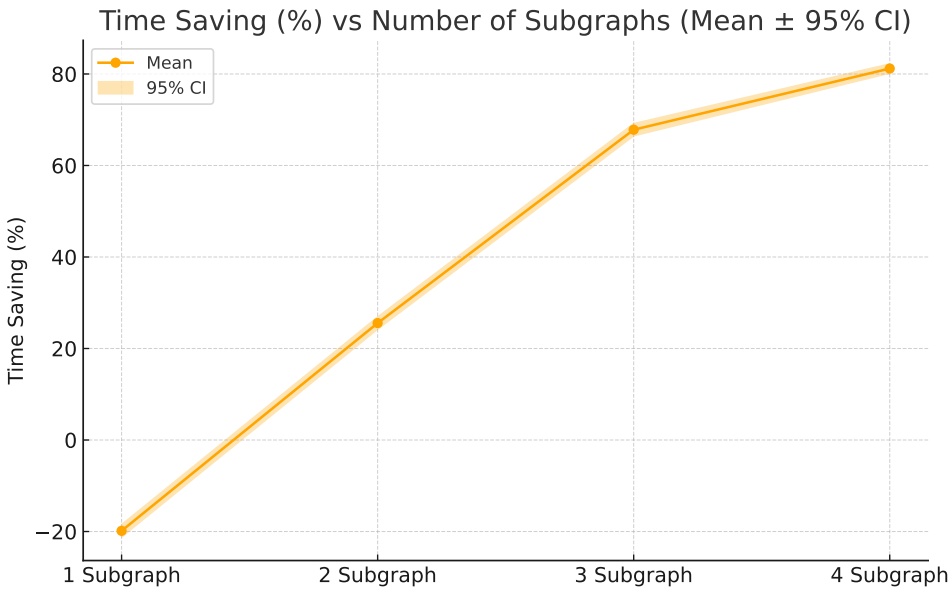


Figure 5.6: Time savings as a function of number of subgraphs

sum of per-subproblem enumerations. As a result, the advantage gained from incorporating additional subgraphs increases at a slower rate.

In scenarios involving strictly independent subgraphs, decomposition serves as an effective scalability strategy. CP-SAT achieves significantly greater and more consistent performance improvements—mean savings increase from approximately 25% ($k = 2$) to 68% ($k = 3$) and 81% ($k = 4$)—due to its robust local pruning capabilities and straightforward parallelization approach. Conversely, DFS continues to be constrained by its reliance on per-subproblem enumeration.

5.5 Phase-wise Runtime Breakdown

To accurately delineate the computational effort expended by each algorithm—and to offer a mechanistic explanation for their differing scalability—we decomposed the total runtime into two primary stages: Step 2 (the solution search for each subgraph) and Step 3 (the global merging and post-processing of results). The measurements, as illustrated as stacked bars in Figure 5.7 and Figure 5.8, underscore the fundamentally distinct computational philosophies underlying the baseline DFS and the CP-SAT solver.

Late-Stage Complexity Bottlenecks in DFS-Based Search

In the DFS pipeline, the initial search conducted within each subgraph (Step 2) is characterized by lightweight recursive enumeration. As this step primarily involves local feasibility checks, its runtime is minimal, visually appearing as a mere sliver in the stacked bars of Figure 5.7(a).

The architecture of the algorithm defers all significant complexity to Step 3, which consequently dominates the overall runtime profile. This stage consists of two parts: first, a merging routine that performs recursive Cartesian products across all solution sets from the subgraphs; second, an expensive filtering procedure designed to enforce global feasibility and eliminate duplicates. Due to the multiplicative growth in size of this intermediate candidate pool, this phase becomes combinatorially explosive and swiftly establishes itself as a computational bottleneck. As vividly illustrated in Figure 5.7(a), the runtime for Step 3 (represented by the orange bar) accounts for over 98% of total execution time while exhibiting exponential growth—exceeding 63 seconds for instances with 16 nodes—and overshadowing all other computational costs.

Figure 5.7 and Figure 5.8: Phase-wise runtime breakdown for (a) DFS and (b) CP-SAT across varying problem sizes. The charts underscore the late-stage explosion observed in the DFS pipeline compared to the front-loaded, search-intensive profile associated with CP-SAT.n at 16 nodes, this stage concludes in milliseconds and remains essentially inconspicuous within the stacked bars presented in Figure 5.8(b).

Early Pruning and Efficient Solution Merging in CPSAT

The CP-SAT pipeline demonstrates a contrasting operational profile. In this framework, computational efforts are strategically concentrated in Step 2. During this phase, the constraint model comprehensively encodes all rules related to node choices, capacities, and compatibility, driven by a clear objective of minimizing the number of additional devices. The solver initially determines the optimal objective value and subsequently enumerates only the unique optimal assignments. This ”online” deduplication process is efficiently managed through a solution callback utilizing canonical resource keys, ensuring that each subgraph generates a compact and highly qualified solution set.

As a result, the intelligent search conducted in Step 2 accounts for the majority of CP-SAT’s runtime—typically ranging from 70% to 90% of the

Table 5.3: Strategic differences between DFS and CPSAT pipelines.

Aspect	**DFS Approach with Deferred Complexity**	**CPSAT Approach with Early Intelligence**
Complexity Management	Defers constraint validation to a final, brute-force merge-and-filter stage, causing a combinatorial explosion.	Integrates optimization, pruning, and uniqueness directly into the initial search phase, preventing explosions.
Deduplication Strategy	Post-mortem: Applies canonical keys to filter a massive, already-generated candidate list.	Online: Applies canonical keys during the solution search to prevent duplicate generation from the outset.
Parallelism	The core merging logic is recursive and inherently sequential, limiting parallel execution benefits.	The subgraph-solving nature of Step 2 is embarrassingly parallel, scaling well with available CPU threads.

total time—as illustrated in Figure 5.8(b). This reflects a deliberate allocation of computational resources towards leveraging the solver's robust propagation and conflict-learning capabilities.

This investment yields significant dividends in Step 3, which is streamlined into an incremental merging process. Given that inputs have already been optimized and deduplicated, this merging stage necessitates only linear passes over compact data structures. Consequently, this phase operates with such efficiency that its execution time is measured in milliseconds; it appears as an almost imperceptible segment on stacked bar representations.

Comparative Insights

The phase-wise analysis provides a definitive explanation for CP-SAT's superior scalability, which can be summarized by three key strategic differences:

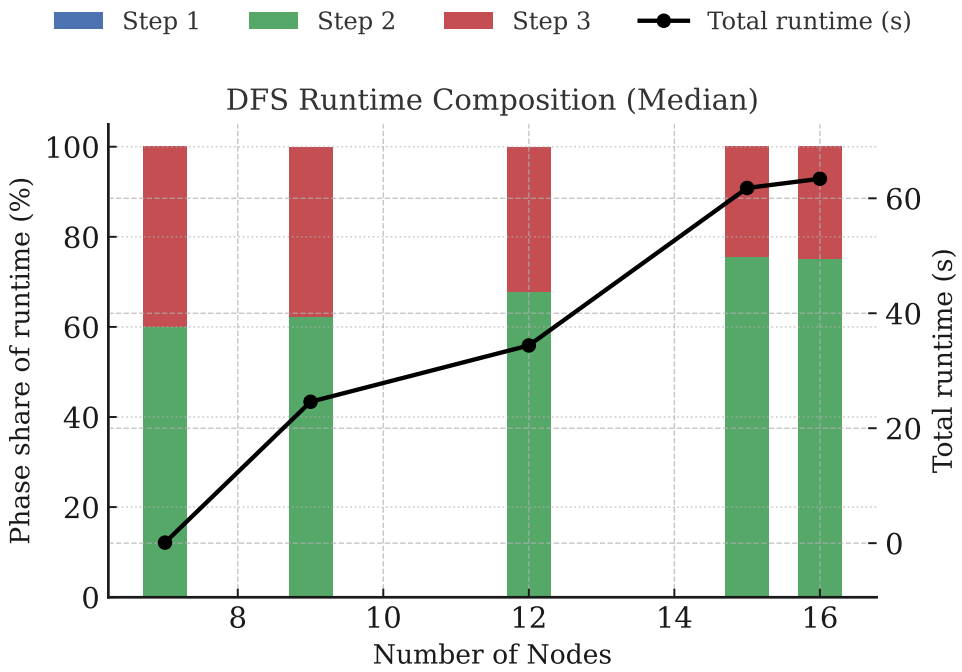


Figure 5.7: Phase-wise runtime composition of the DFS baseline, showing how the relative contributions of Step 1, Step 2, and Step 3 evolve with increasing node count, together with the corresponding total runtime indicated by the black line

5.6 Summary of Results

The experimental evaluation demonstrates that the CP-SAT-based approach consistently outperforms the DFS baseline across multiple dimensions, including correctness, runtime efficiency, memory consumption, and scalability. These findings highlight the structural advantages of constraint programming techniques over procedural enumeration in complex orchestration tasks.

In terms of correctness, CP-SAT is able to generate complete and valid sets of optimal solutions for all test cases. In contrast, DFS fails to do so in several scenarios. Specifically, it exhibits incorrect behavior when dealing with cyclic topologies such as ring networks, often missing feasible cyclic paths. Additionally, in scenarios involving shared resource constraints, DFS omits valid combinations due to flawed backtracking logic. These failures are not isolated to specific configurations, but rather reflect a general inability of the DFS approach to handle topological and logical complexity.

Regarding runtime performance, CP-SAT consistently exhibits better

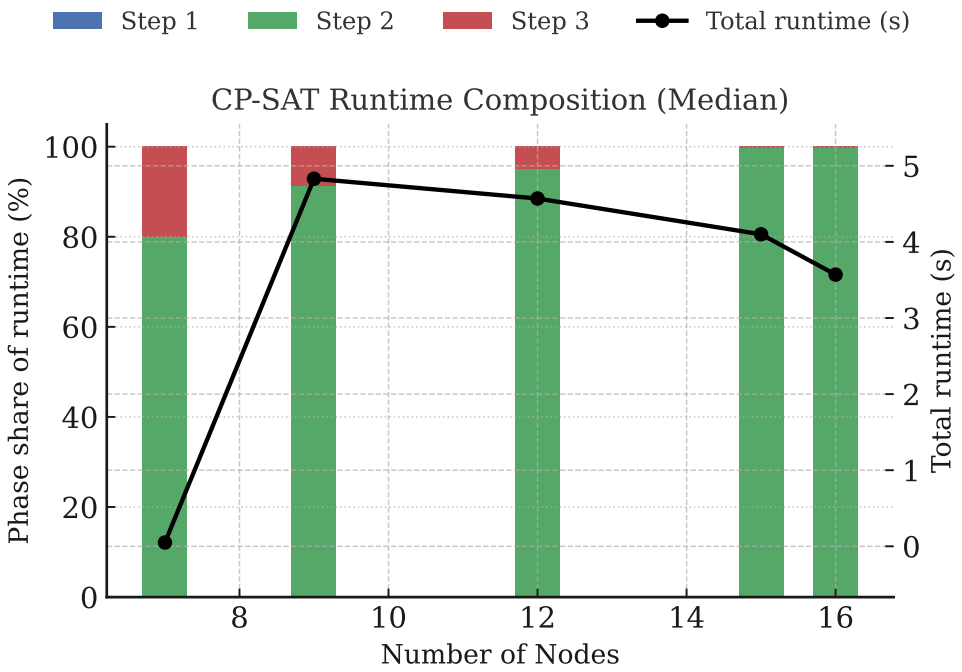


Figure 5.8: Phase-wise runtime composition of the CP-SAT solver, highlighting the dominance of Step 2 in the overall solving process and the comparatively stable total runtime across different node sizes

scalability. On average, it reduces computation time by approximately 25 percent across the full set of benchmarks. In the most computationally demanding cases, it achieves reductions of up to 90 percent. This improvement is particularly evident in instances with a large number of valid solutions, where the runtime of DFS grows superlinearly, approaching quadratic behavior. CP-SAT, on the other hand, maintains near-linear runtime even as the number of optimal solutions increases into the thousands. The relative speedup becomes more pronounced in larger instances, with CP-SAT often outperforming DFS by a factor exceeding eight, and in some cases reaching speedups of over seventeen times.

Memory usage is also significantly lower with CP-SAT, particularly for large problem instances. While both solvers require comparable memory for small-scale graphs, DFS incurs substantial overhead due to recursive call stacks, which grow rapidly with the size and branching of the search space. In contrast, CP-SAT maintains a controlled and predictable memory footprint, reducing the average peak memory usage by more than 12 percent and exceeding 50 percent savings in some extreme cases.

Scalability is further demonstrated through decomposition experiments. When the problem graph is partitioned into multiple independent components, the CPSAT-based solver achieves considerable time savings. The experimental data show mean runtime reductions of 25 percent for two subcomponents, 68 percent for three, and 81 percent for four. The performance gain increases with the number of subgraphs, reflecting CPSAT's ability to exploit parallelism and prune each subproblem efficiently. DFS, in contrast, derives little benefit from decomposition, as it lacks the structural mechanisms to parallelize or pre-prune subgraphs effectively.

The underlying architectural differences explain the observed performance gap. DFS postpones constraint checking and duplicate elimination to a late stage, requiring extensive brute-force merging and filtering after candidate solutions have been generated. This results in a back-loaded computation pipeline that concentrates most of the runtime in a costly and inefficient post-processing phase. CP-SAT, by comparison, integrates constraint propagation, search pruning, and deduplication directly into the solving process. This allows the solver to reduce the solution space during the search itself, rather than after it is exhaustively enumerated, leading to much more compact and efficient computation.

In summary, the empirical results strongly support the conclusion that constraint programming using the CP-SAT solver offers superior correctness, efficiency, and scalability for orchestration problems characterized by complex resource constraints and connectivity requirements. The combination of early pruning, declarative modeling, and parallel solving positions CP-SAT as a practical and robust choice for real-world deployment in testbed orchestration systems.

5.7 Reliability Analysis

To ensure the reliability of our findings, all experiments were conducted 20 times under identical conditions. The resulting variance was minimal: the 95% confidence intervals for runtime savings (Figure 5.6) are narrow, and the boxplots of memory usage (Figure 5.5) demonstrate tightly clustered distributions. This statistical stability confirms that the observed performance differences are not artifacts of noise but rather systematic consequences of the algorithmic strategies employed.

The CP-SAT method utilizes Google's production-grade OR-Tools solver, a well-established and extensively validated library. Its correctness and reproducibility are ensured through constraint propagation and conflict

learning, which deterministically eliminate infeasible search regions. In contrast, the correctness issues associated with DFS—particularly in ring topologies and scenarios involving sharable resources—were consistently replicated across all runs, highlighting these as inherent limitations of this approach. While alternative implementations of DFS may exhibit variations in engineering details, they cannot fundamentally address these correctness and scalability constraints.

5.8 Validity Analysis

The validity of this comparative study is supported by three dimensions: internal validity, external validity, and construct validity. Each dimension is addressed to ensure methodological rigor and trustworthy conclusions.

Internal validity refers to the extent to which observed performance differences can be attributed to the algorithms themselves. To ensure a fair comparison, all experiments were conducted in a consistent and controlled computational environment. Hardware configurations, operating system versions, and input data were held constant across all runs. This setup guarantees that the only independent variable influencing outcomes is the choice of solving algorithm.

External validity concerns the generalizability of the findings. The evaluation includes a comprehensive set of test cases with varied graph topologies, including linear chains, cyclic rings, barbell-shaped graphs, and hub-and-spoke configurations. This diversity reduces the likelihood of overfitting results to a narrow class of problems and enhances the applicability of conclusions to broader real-world network scenarios.

Construct validity assesses whether the selected metrics accurately reflect the intended evaluation goals. Metrics such as runtime and peak memory usage correspond to computational efficiency and resource consumption, respectively, while the completeness of the solution set captures the solver's ability to enumerate all valid optimal configurations. In addition, step-wise runtime breakdowns (see Figure **??**) offer further insight into solver behavior. For example, the DFS baseline delays most of the computational complexity until the merging phase, where combinatorial explosion may occur. In contrast, the CP-SAT solver employs early constraint propagation during the solving phase, which reduces post-processing time and improves efficiency. These measures are aligned with the core research questions and provide a valid foundation for comparison.

Chapter 6

Discussion

The empirical results presented in the previous chapter not only quantify the performance advantage of the CP-SAT solver but also highlight a deeper divergence in computational paradigms for solving constrained optimization problems. This section interprets the key findings, discusses their broader significance for both engineering practice and academic research, and reflects on the limitations of the current study.

A central conclusion of this work is that the performance gap between the CP-SAT solver and the baseline DFS algorithm is not merely a matter of execution speed. Rather, it reflects a fundamental shift from an approach based on exhaustive enumeration and post hoc filtering to one that leverages intelligent, constraint-guided pruning from the outset.

The observed shortcomings of the DFS baseline in handling ring topologies and sharable resources are not attributable to a specific implementation flaw. Instead, they reveal intrinsic limitations of procedural search strategies. In such approaches, developers must manually manage system state and enforce complex constraints, a task that becomes increasingly difficult as problem complexity grows. This often results in failures to guarantee solution correctness or completeness. The phase-wise breakdown of execution time further explains this outcome: DFS initially performs a fast but naïve search, producing a large volume of candidate solutions. The computational burden is thus postponed to a later merging and filtering phase, which suffers from combinatorial explosion. This pattern represents an accumulation of what may be described as computational debt, which must eventually be resolved at a high cost during post-processing.

In contrast, the CP-SAT solver invests computational effort earlier in the solving process. During the initial constraint-based search phase, the

solver applies advanced techniques such as constraint propagation and conflict learning to eliminate infeasible regions of the search space. Although this phase appears computationally intensive, it produces a compact set of high-quality, optimal solutions that greatly reduce the cost of subsequent merging steps. This strategy avoids deferred complexity and leads to more scalable and reliable performance.

The implications of these findings extend beyond the specific problem studied. For engineering practitioners, the results suggest that modern declarative constraint solvers offer a practical and efficient alternative to custom procedural algorithms for tasks such as scheduling, planning, and resource allocation. Modeling constraints declaratively, rather than implementing control logic manually, improves maintainability and reduces the risk of implementation errors. It also mitigates long-term technical debt and contributes to more robust software systems.

From a research perspective, this study demonstrates that general-purpose solvers are capable of outperforming specialized algorithms, even in domain-specific contexts. It illustrates the growing maturity of constraint programming tools and their applicability to real-world NP-hard problems. Moreover, it points to a democratization of optimization technologies, where powerful solving capabilities become accessible to practitioners beyond the traditional operations research community.

Chapter 7

Conclusions and Future work

This chapter synthesizes the outcomes of the degree project, reflecting upon the achieved goals and the insights gained. It proceeds to outline the limitations of the current work and proposes concrete directions for future research.

7.1 Conclusions

This thesis investigated an efficient and reliable approach to solving a complex resource combination problem in the context of network orchestration, with the goal of evaluating its performance against a baseline depth-first search (DFS) implementation. The findings of this study support several key conclusions.

The first objective of this work was to assess whether a constraint programming-based approach could provide tangible benefits over procedural methods. This goal was achieved. The CP-SAT solver, part of Google's OR-Tools framework, consistently outperformed the DFS baseline across all major evaluation criteria, including correctness, runtime efficiency, memory usage, and scalability. These results validate the effectiveness of declarative optimization techniques in this domain.

The most significant insight gained from this study concerns the contrasting philosophies of the two algorithmic approaches. Procedural search methods such as DFS often struggle with complex global constraints because they defer most of the computational burden to later phases of execution. In this case, DFS produced a large number of candidate solutions early on, only to encounter bottlenecks during the merging and filtering steps due to combinatorial explosion. In contrast, the constraint programming solver was

able to eliminate infeasible regions of the search space early in the process through propagation and constraint reasoning. This difference in strategy—pruning infeasibility at the source rather than filtering after the fact—proved to be a key factor in the CP-SAT solver's superior performance.

From a methodological standpoint, this study highlights the value of declarative modeling in tackling real-world optimization problems. For researchers working on similar challenges, it is strongly recommended to adopt solver frameworks such as OR-Tools early in the project lifecycle. Declarative models separate problem formulation from solving strategy, allowing researchers to focus on expressing the core logic of the problem. This not only improves correctness and performance but also results in more maintainable and extensible codebases. While the initial learning curve may seem steep, the long-term benefits in clarity and efficiency are substantial.

Reflecting on the development process, it is apparent that earlier identification of the DFS baseline's limitations could have improved the research workflow. A significant portion of time was dedicated to refining a procedural algorithm that, in hindsight, was not well suited to the structure of the problem. More rapid prototyping and earlier comparative evaluation might have enabled a quicker transition toward the more promising constraint programming direction. With more time allocated to this solver, additional techniques—such as customized search heuristics or multi-objective extensions—could have been explored to further enhance solution quality and flexibility.

In conclusion, this research affirms the advantages of modern constraint programming solvers for orchestrating complex resource allocation tasks and contributes a generalizable methodology that can be applied to a variety of similar optimization problems.

7.2 Limitations

While the results presented in this study demonstrate the clear advantages of constraint programming over procedural search in the tested scenarios, several limitations must be acknowledged to properly contextualize these findings.

First, the conclusions are based on a specific formulation of a resource allocation and orchestration problem. Although the test suite included a variety of topological patterns, such as chains, rings, and star-shaped graphs, the generalizability of the results to unrelated combinatorial domains remains uncertain. In problems characterized by minimal constraints—such as simple pathfinding or local scheduling—lightweight procedural methods may

perform comparably or even outperform constraint solvers due to their lower overhead.

Second, the internal mechanisms of the constraint programming solver, particularly the CP-SAT solver used in this study, are largely opaque. While the solver offers strong performance and flexibility, it also encapsulates complex search heuristics, presolve techniques, and propagation strategies that are not easily exposed or adjusted by the user. This can present challenges in debugging, performance tuning, and especially in applications that require a high level of transparency or explainability. Unlike procedural methods where each decision is explicitly coded and observable, it can be difficult to trace why one solution is preferred or how the solver arrived at a specific decision path.

Third, the experimental scale remains limited. Although the solver demonstrated good scalability across increasing problem sizes within the testbed, experiments were not conducted on full-scale production datasets, which may involve hundreds or thousands of nodes and resource groups. At such scales, new technical issues may arise. These include a significantly increased memory footprint, longer presolve times, or difficulties in efficiently enumerating all optimal solutions due to the combinatorial explosion in solution space.

In summary, while the proposed method offers strong advantages in expressiveness, correctness, and robustness for mid-sized orchestration problems, its applicability to simpler tasks or extremely large-scale industrial settings may require further investigation and validation.

7.3 Future work

Given the breadth of the problem, this project serves as a strong foundation, but several avenues remain for future exploration. The following sections outline some of the remaining issues that warrant attention in subsequent work.

7.3.1 Unaddressed Aspects of the Problem

The current prototype successfully addresses the static resource allocation problem. A notable unaddressed requirement, however, is the handling of dynamic updates, such as the real-time addition or removal of available resources without necessitating a full re-solve. In a real-world operational scenario, a piece of equipment might fail and be taken offline, and the system must be capable of reacting swiftly. This dynamic aspect remains an open problem.

7.3.1.1 Cost and Multi-Objective Analysis

The current model optimizes solely for the number of extra devices. A practical, industry-relevant solution would necessitate a more sophisticated cost analysis, incorporating factors such as equipment capital expenditure, power consumption, or signal latency as secondary objectives. Future work must therefore extend the CP-SAT model to handle multi-objective optimization. Such an extension would not yield a single best solution, but rather a Pareto front—a set of non-dominated solutions from which a human decision-maker can select the optimal trade-off based on business priorities.

7.3.1.2 Integration into a Dynamic System

The current solution operates in a batch mode on a static set of inputs. A crucial subsequent step is to investigate its integration into a dynamic system where constraints can change in real-time. This would require the exploration of techniques for incremental solving or warm-starting the solver. A warm start, which utilizes a previous optimal solution as a heuristic guide for the new search, is often dramatically faster than solving from scratch and represents a key technique for making optimization practical in dynamic environments.

7.3.2 Immediate Directions for Subsequent Research

In particular, it is asserted that handling dynamic constraints remains the most critical problem to be solved for this solution to be viable in a production environment; its resolution should be the next research priority. A promising direction for subsequent researchers is an investigation into the ”assumptions” feature within SAT solvers. This powerful capability allows for the temporary imposition of constraints (e.g., ”assume resource X is unavailable”) to test for feasibility without the high overhead of rebuilding the entire model. This functions as a form of ”what-if” analysis and presents an ideal, highly efficient approach for managing dynamic resource availability.

References

[1] W. Azariah, F. A. Bimo, C.-W. Lin, R.-G. Cheng, N. Nikaein, and R. Jana, "Open radio access networks," Encyclopedia (MDPI) Entry, 2024, accessed: 02 September 2025. [Online]. Available: https://encyclopedia.pub/entry/55892

[2] J. L. N. Frauendorf and E. Souza, "The evolution of ran (radio access network), d-ran, c-ran, v-ran, and o-ran," in *The Architectural and Technological Revolution of 5G*. Springer, 2022.

[3] N. Ejaz and S. Choudhury, "A comprehensive survey of linear, integer, and mixed-integer programming approaches for optimizing resource allocation in 5g and beyond networks," *arXiv preprint*, vol. arXiv:2502.15585, 2025, cs.NI. [Online]. Available: https://arxiv.org/abs/2502.15585

[4] L. Perron and F. Didier, "The CP-SAT primer: Using and understanding Google OR-Tools' CP-SAT solver," Google OR-Tools Documentation, May 2024, accessed: 13 September 2025. [Online]. Available: https://developers.google.com/optimization/cp/cp_sat

[5] D. Qi, S. Shen, and G. Wang, "Towards an efficient vnf placement in network function virtualization," *Computer Communications*, vol. 138, no. 3, pp. 1–6, 2019. doi: 10.1016/j.comcom.2019.03.005. [Online]. Available: https://doi.org/10.1016/j.comcom.2019.03.005

[6] J. N. Hooker and W.-J. van Hoeve, "Constraint programming and operations research," *Constraints*, vol. 23, no. 2, pp. 172–195, 2018. doi: 10.1007/s10601-017-9280-3. [Online]. Available: https://doi.org/10.1007/s10601-017-9280-3

[7] R. Mijumbi, J. Serrat, J.-L. Gorricho, N. Bouten, F. De Turck, and R. Boutaba, "Network function virtualization: State-of-the-art

and research challenges," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 1, pp. 236–262, 2016. doi: 10.1109/COMST.2015.2477041. [Online]. Available: https://ieeexplore.ieee.org/document/7243304

[8] Y. Xu, G. Gui, H. Gacanin, and F. Adachi, "A survey on resource allocation for 5g heterogeneous networks: Current research, future trends, and challenges," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 2, pp. 668–695, 2021. doi: 10.1109/COMST.2021.3059896. [Online]. Available: https://doi.org/10.1109/COMST.2021.3059896

[9] W. Ejaz, S. Sharma, M. Bennis, and E. Hossain, "A comprehensive survey on resource allocation for cran in 5g and beyond networks," *Computer Networks*, vol. 183, no. C, pp. 1–18, 2020. doi: 10.1016/j.comnet.2020.03.012. [Online]. Available: https://doi.org/10.1016/j.comnet.2020.03.012

[10] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "Understanding o-ran: Architecture, interfaces, algorithms, security, and research challenges," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 3, pp. 1546–1607, 2023. doi: 10.1109/COMST.2023.3239220. [Online]. Available: https://doi.org/10.1109/COMST.2023.3239220

[11] A. Laghrissi and T. Taleb, "A survey on the placement of virtual resources and virtual network functions," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1409–1434, 2018. doi: 10.1109/COMST.2018.2884835. [Online]. Available: https://doi.org/10.1109/COMST.2018.2884835

[12] D. Bhamare, R. Jain, M. Samaka, and A. Erbad, "A survey on service function chaining," *Journal of Network and Computer Applications*, vol. 75, pp. 138–155, 2016. doi: 10.1016/j.jnca.2016.09.001. [Online]. Available: https://doi.org/10.1016/j.jnca.2016.09.001

[13] B. Yi, X. Wang, S. K. Das, K. Li, and M.-s. Huang, Member, "A comprehensive survey of network function virtualization," *Computer Networks*, vol. 133, no. C, pp. 212–262, 2018. doi: 10.1016/j.comnet.2018.01.021. [Online]. Available: https://doi.org/10.1016/j.comnet.2018.01.021

[14] T. Liu, F. Callegati, W. Cerroni, C. Contoli, M. Gabbrielli, and S. Giallorenzo, "Constraint programming for flexible service

function chaining deployment," in *Proceedings of the 52nd Hawaii International Conference on System Sciences (HICSS)*, 2019. doi: 10.48550/arXiv.1812.05534 pp. 2004–2013. [Online]. Available: https://arxiv.org/abs/1812.05534

[15] M. F. Bari, S. R. Chowdhury, R. Ahmed, and R. Boutaba, "On orchestrating virtual network functions in nfv," in *Proceedings of the 11th IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2015. doi: 10.1109/NOMS.2016.7502825 pp. 192–200. [Online]. Available: https://doi.org/10.1109/NOMS.2016.7502825

[16] E. Amaldi, M. Chiarandini, and A. Marchetti-Spaccamela, "On the computational complexity of the virtual network embedding problem," *Theoretical Computer Science*, vol. 609, no. C, pp. 45–61, 2016. doi: 10.1016/j.tcs.2016.03.021. [Online]. Available: https://doi.org/10.1016/j.tcs.2016.03.021

[17] N. Ejaz and S. Choudhury, "A comprehensive survey of linear, integer, and mixed-integer programming approaches for optimizing resource allocation in 5g and beyond networks," arXiv preprint arXiv:2502.15585, 2025, submitted 21 February 2025. [Online]. Available: https://arxiv.org/abs/2502.15585

[18] L. Ruiz, R. J. Durán, I. De Miguel, P. S. Khodashenas, J.-J. Pedreño-Manresa, N. Merayo, J. C. Aguado, P. Pavón-Marino, S. Siddiqui, J. Mata, P. Fernández, R. M. Lorenzo, and E. J. Abril, "A genetic algorithm for vnf provisioning in nfv-enabled cloud/mec ran architectures," *Applied Sciences*, vol. 8, no. 12, p. 2614, 2018. doi: 10.3390/app8122614. [Online]. Available: https://doi.org/10.3390/app8122614

[19] M. C. Luizelli, W. L. da Costa Cordeiro, L. S. Buriol, and L. Paschoal Gaspary, "A fix-and-optimize approach for efficient and large scale virtual network function placement and chaining," *Computer Communications*, vol. 102, no. C, pp. 67–77, 2017. doi: 10.1016/j.comcom.2016.11.002. [Online]. Available: https://doi.org/10.1016/j.comcom.2016.11.002

[20] M. C. Luizelli, L. R. Bays, L. S. Buriol, M. P. Barcellos, and L. P. Gaspary, "Piecing together the nfv provisioning puzzle: Efficient placement and chaining of virtual network functions," in *Proceedings*

of the IFIP/IEEE International Symposium on Integrated Network Management (IM), 2015. doi: 10.1109/INM.2015.7140281 pp. 98–106. [Online]. Available: https://doi.org/10.1109/INM.2015.7140281

[21] Purnawansyah and Haviluddin, "K-means clustering implementation in network traffic activities," in *2016 International Conference on Information and Communication Technology (ICoICT)*, 2016. doi: 10.1109/ICoICT.2016.7892566 pp. 1–6. [Online]. Available: https://ieeexplore.ieee.org/document/7892566

[22] K. Wang, W. Zhou, and S. Mao, "Load-aware dynamic rrh assignment in cloud radio access networks," in *Proceedings of IEEE Wireless Communications and Networking Conference (WCNC)*, 2016. doi: 10.1109/WCNC.2016.7564824 pp. 1–6. [Online]. Available: https://doi.org/10.1109/WCNC.2016.7564824

[23] N. C. Luong, D. T. Hoang, S. Gong, D. Niyato, P. Wang, and Y.-C. Liang, "Applications of deep reinforcement learning in communications and networking: A survey," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 4, pp. 3133–3174, 2019. doi: 10.1109/COMST.2019.2916583. [Online]. Available: https://doi.org/10.1109/COMST.2019.2916583

[24] R. Mohamed, M. Avgeris, A. Leivadeas, and I. Lambadaris, "Fragmentation-aware vnf placement: A deep reinforcement learning approach," in *Proceedings of the IEEE International Conference on Communications (ICC)*. Denver, CO, USA: IEEE, 2024. doi: 10.1109/ICC51166.2024.10623068 pp. 1–6. [Online]. Available: https://doi.org/10.1109/ICC51166.2024.10623068

All SDGs are within the range [1-17].

TRITA – EECS-EX-2026:27
Stockholm, Sweden 2026

www.kth.se